INFORMATION TECHNOLOGY

2026 EDITION

Designing, Building, and Evaluating Digital Systems

A principled introduction to information technology, digitalisation, and application development

Concepts, methods, and practice for digital systems

Scott Brodie Forsyth

TEXTBOOK EDITION

Scott Brodie Forsyth

Designing, Building, and Evaluating Digital Systems

Scott Brodie Forsyth

Preface

Digital systems are made of code, data, interfaces, organisations, and decisions. A program can be syntactically correct and still solve the wrong problem. A database can be fast and still encode a harmful policy. A beautiful interface can be inaccessible. A predictive model can be accurate on a test set and still be unsafe in use. This book is organised around those connections.

The intended reader may arrive from computer science, social science, design, teaching, business, or another discipline. The first chapters therefore begin with ordinary examples and gradually introduce formal language. The aim is not to lower the intellectual level. It is to make the reasoning visible before asking the reader to manipulate its notation.

The scope of this book was inspired by the broad range of questions encountered in a modern information-technology degree, but the book is not a degree programme, course catalogue, or institutional manual. It is a standalone synthesis. Its purpose is to show how ideas from computing, software engineering, interaction design, organisations, research, and artificial intelligence fit together when we try to understand and build digital systems.

One recurring case, CivicQueue, follows a fictional public-service appointment system from an uncertain problem to a tested prototype, a relational database, a deployable web service, an organisational intervention, and a bounded research evaluation. The case is deliberately imperfect. It gives readers opportunities to identify assumptions, privacy risks, competing stakeholder interests, and claims that the evidence cannot support.

The central question is simple: *what must be true for this digital system to be a responsible answer to this problem?* The chapters provide different ways of answering it: computational models, algorithms, software architecture, interaction design, organisational analysis, empirical research, and critical reflection. You can read the book from beginning to end, or enter through the chapter that matches the problem in front of you.

July 2026

How to use this book

Each chapter moves from a concrete problem to concepts, formal reasoning, implementation, limitations, and exercises. Read the first explanation quickly, then slow down when the notation or code appears. A useful test of understanding is whether you can explain the idea without repeating the book's example.

The recurring case is not a single prescribed project. Treat it as a laboratory. Replace its public-service setting with a domain you know, but preserve the discipline of stating stakeholders, assumptions, data, interfaces, failure modes, and evidence.

A practical study loop

1. Read the motivating problem and write down what is still unknown.
2. Run the smallest executable example before changing it.
3. Draw the system or model in your own notation.
4. Solve the conceptual exercises without looking at the solutions.
5. Modify the example to violate one assumption and observe the failure.
6. Record the decision, evidence, and limitation in a project log.

The exercises are grouped as conceptual, technical, design, programming, research, and mini-project work. In collaborative work, distribute responsibility for implementation, modelling, user research, testing, and writing, but rotate roles so that everyone can explain the whole system and its limitations.

> **Caution.** A passing test suite is evidence about the tested cases. It is not proof that the system is correct in every environment. The same distinction applies to user studies, simulations, and machine-learning evaluations.

How the book is organised

This book is a guided journey through the ideas needed to understand, design, build, and evaluate digital systems. It is not a timetable or a list of institutional requirements. The chapters are arranged so that each part supplies concepts that make the next part easier, while still allowing readers to move directly to a topic that matters to them.

Table 1: The architecture of the book.

Part or focus	**Chapters**	**Central questions**	**What the reader develops**
Foundations	1–4	What is a digital system? How can a problem be represented computationally?	Problem statements, models, programs, algorithms, and computational reasoning
Engineering systems	5–10	How can software be designed, implemented, stored, tested, secured, and operated?	Requirements, architecture, code, data, testing, security, and operational judgment
People and practice	11–14	How do people experience systems, and how do organisations and teams shape them?	Interaction design, evaluation, organisational analysis, agile practice, and collaboration
Evidence and independent work	15–18	How can claims be investigated, evaluated, and connected to responsible action?	Research design, human-centred AI, experimentation, entrepreneurship, and thesis reasoning
Integration projects	19–20	What happens when the concepts meet in a complete problem?	A worked case, project ideas, evaluation plans, and final synthesis
Appendices and tools	A–D	Which mathematical, technical, and conceptual tools support the main chapters?	Mathematics, Python and Git, reference tables, and a glossary

The parts are not prerequisites in the bureaucratic sense, and they are not meant to be completed at a fixed speed. A beginner may read sequentially and run the examples as they appear. An experienced reader may begin with the case study, research chapter, security material, or a specific engineering problem, then return to earlier chapters when a foundation is needed.

The recurring movement

The book repeatedly moves through a chain of questions:

problem → model → program → component → system → organisation → evidence.

This is not a one-way lifecycle. User research can change the problem; production incidents can change the model; organisational constraints can change the architecture; and evidence can show that an intervention did not achieve its intended effect. The purpose of the sequence is therefore not to prescribe a recipe, but to make the connections visible.

Prerequisite bridge

This short bridge is for readers who have not studied computer science. You need only ordinary arithmetic, careful reading, and the willingness to test claims.

Numbers and units

Computers represent quantities using finite strings of bits. A bit has two possible states, conventionally written 0 and 1. Eight bits form a byte. A kilobyte is used in two different ways: decimal SI usage means 1,000 bytes, while the binary unit kibibyte means 1,024 bytes. Technical writing should state which convention it uses.

Functions as input-output rules

In mathematics, a function assigns exactly one output to each allowed input. If $f(x) = 2x+1$, then $f(3) = 7$. In programming, a function may also change state, perform I/O, raise an exception, or return different results because of time or randomness. The mathematical analogy is useful, but it is not a complete definition of a program function.

Evidence and uncertainty

A measurement is an observation produced by a procedure. It can be noisy, biased, incomplete, or affected by the act of measurement. A sample is not automatically representative of a population. A result can be compatible with a hypothesis without proving it. These distinctions reappear in usability studies, organisational evaluation, and machine learning.

The minimum command-line vocabulary

The command line is a text interface for starting programs and inspecting files. A command has a current working directory, inputs, outputs, and an exit status. Version control records changes to files over time. The appendix introduces both tools carefully; the early chapters use commands only when they illuminate the program.

> **In plain language.** A technical system is easier to understand when you can state what goes in, what comes out, what changes in between, and what would count as failure. That is the habit this bridge is designed to establish.

Contents

List of Figures

Part I

Foundations: problems, models, and programs

Chapter 1

Information technology as a way of shaping action

1.1 A motivating problem

Imagine a municipality that wants to reduce missed appointments at a public service centre. Someone proposes an app that lets citizens book, reschedule, and receive reminders. The proposal sounds technical, but the problem is not only technical. Are appointments currently missed because people forget, because the booking process is confusing, because transport is unreliable, or because the available times conflict with work? Who may see a citizen's case? What happens when a person has no smartphone? Which employee is responsible when two systems disagree?

The example is the starting point for CivicQueue, the recurring case in this book. The application matters, but so do the policy, the work routines, the data definitions, the interface, the organisational incentives, and the evidence used to decide whether the intervention helped.

> **What you will learn.** By the end of this chapter, you should be able to distinguish computer science, software engineering, information systems, HCI, and digitalisation; describe a digital system as a technical and sociotechnical object; identify boundaries, stakeholders, assumptions, and failure modes; and formulate an initial problem statement suitable for a real project.

1.2 The five-minute explanation

Information technology is the study and practice of using representations, computation, communication, and automated action to support human and organisational purposes. A computer is not the purpose. It is a general mechanism for manipulating representations according to rules.

> **In plain language.** If a paper form is replaced by a web form, that is digitisation: a change in medium. If the organisation changes how cases are allocated, who can act, what evidence is collected, and how citizens participate, that is digitalisation: a change in practice enabled by digital technology. The two often occur together, but they are not the same.

Four perspectives help us ask different questions:

Computer science What can be computed, how can it be represented, and how efficiently and correctly can it be done?

Software engineering How can a team produce and maintain software that is reliable, secure, usable, and adaptable under constraints?

Information systems How do technology, people, processes, rules, and organisations combine to create or fail to create value?

Human–computer interaction How do people understand, use, experience, and appropriate interactive systems, and how should those systems be designed and evaluated?

These are not competing definitions of the same job. They are complementary lenses. A system can be computationally efficient but organisationally useless, socially harmful, or impossible for users with disabilities.

1.3 From a problem to a system

A useful first move is to separate five statements that are often collapsed into one:

1. **Situation:** appointments are missed more often than the organisation considers acceptable.
2. **Interpretation:** reminders and flexible rescheduling may address part of the problem.
3. **Intervention:** build CivicQueue with a reminder and rescheduling workflow.
4. **Mechanism:** the workflow reduces forgetting or lowers the cost of changing an appointment.
5. **Outcome claim:** after adoption, missed appointments decrease without unacceptable exclusion or administrative burden.

The fifth statement is empirical. It cannot be established merely by demonstrating that the application runs.

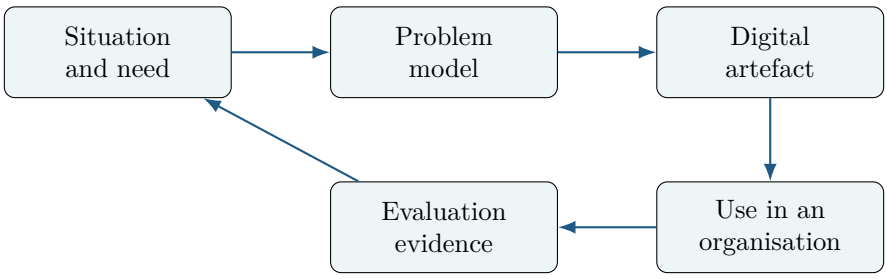

Figure 1.1: A digital intervention is a cycle of interpretation, construction, use, and evidence.

The arrows in Figure 1.1 are not a rigid waterfall. New evidence can change the problem model; implementation constraints can reveal that the proposed intervention is infeasible; use can produce unexpected workarounds.

1.4 A formal vocabulary for boundaries

For a first approximation, model a system as a tuple

$$S = (I, O, X, T, R),$$

where I is a set of inputs, O a set of outputs, X a set of internal states, T a transition rule, and R a set of relevant constraints or requirements. An input might be a request to reschedule. A state might record an appointment and its status. The transition rule determines which state follows a request. An output might be a confirmation page or a message to a staff system.

This model is useful because it forces us to ask what changes and what is observed. It is incomplete because real systems also include people, institutions, physical infrastructure, incentives, and interpretations. In CivicQueue, a transition such as "appointment confirmed" is not merely a database update: it may trigger a legal obligation, a staff workflow, and a citizen expectation.

1.5 History and intellectual context

The modern field grew from several traditions: mathematical logic and computation, engineering of reliable artefacts, studies of organisations and information, and research into human action and communication. Turing's analysis of computation helped establish questions about what a machine can do and how we should describe machine behaviour [**?**]. Later, the expansion of networked computing made the boundary between a software product and an organisation increasingly porous.

The history matters because terms such as "automation" and "intelligence" carry assumptions. A technical system does not become neutral merely because its rules are expressed in code. Choices about categories, defaults, thresholds, and access rights are policy choices as well as implementation choices.

1.6 Working example: a bounded problem statement

An unhelpful project statement is: "Build an app for appointments." It identifies a solution before establishing the problem. A stronger initial formulation is:

> How can a public-service centre support citizens in changing appointments without telephone contact, while preserving accessibility, privacy, staff control, and a reliable record of appointment status?

The question is still broad, but it identifies an activity, a context, constraints, and a boundary. It does not promise that a digital channel will reduce missed appointments. That is a claim to investigate.

1.6.1 Stakeholders and system boundary

Primary stakeholders include citizens, front-desk staff, case workers, managers, and system administrators. Secondary stakeholders include accessibility advisers, data-protection officers,

the organisation that supplies the calendar service, and people affected by resource allocation but not directly using the app. A boundary diagram should distinguish:

- the CivicQueue application;
- external identity, calendar, and notification services;
- the municipality's case system;
- people and routines that interpret the outputs.

Caution. A system boundary is an analytical decision, not a fact discovered in the code. If the boundary excludes the call centre, language support, or the notification provider, the project may hide important causes of failure.

1.7 Professional and ethical responsibility

The developer's responsibility is not exhausted by implementing the requested feature. Questions include data minimisation, meaningful consent, retention, access control, accessibility, error recovery, vendor dependency, and the consequences of exclusion. The GDPR provides a legal framework for personal-data processing, but compliance is not identical to ethical adequacy [**?**].

A responsible project makes uncertainty visible. It documents which requirements came from stakeholders, which are assumptions, which are design decisions, and which were evaluated. It gives users a path to recover from mistakes and a human route when the digital path fails.

1.8 Using the chapter in a project

Start a project notebook with four tables:

1. situation and problem evidence;
2. stakeholder interests and risks;
3. assumptions and open questions;
4. proposed artefacts and evidence needed to evaluate them.

When documenting the work, separate the problem analysis from the solution proposal. When presenting it, be ready to explain why the boundary is reasonable, which stakeholder is missing, and what evidence would change your interpretation.

1.9 Summary and key terms

Information technology joins computation, artefacts, people, and institutions. Computer science, software engineering, information systems, and HCI answer different but connected questions. Digitalisation changes practices and relations, not just file formats. A running application is an intervention, not automatically a solution or a research result.

Key terms. information technology; digitisation; digitalisation; computer science; software engineering; information systems; HCI; stakeholder; system boundary; sociotechnical system; requirement; intervention; mechanism; outcome.

1.10 Exercises and worked solutions

Exercise. Choose a familiar service. Write one sentence describing its situation, one sentence describing a possible intervention, and one sentence describing an outcome that would require evidence.

Worked solution. For a library, the situation might be that reserved books are not collected in time. The intervention might be a reminder and cancellation workflow. The outcome claim could be that the proportion of reservations collected within seven days increases without disadvantaging users who do not use email. The three sentences are deliberately different kinds of statement.

Exercise. Which perspective is most directly concerned with a confusing rescheduling screen: computer science, software engineering, information systems, or HCI? Which other perspectives still matter?

Worked solution. HCI is the most direct perspective because it concerns interaction and understanding. Software engineering matters because the screen must be maintainable and integrated with reliable state transitions; computer science matters for the underlying computation; information systems matters because the screen changes a work process and may redistribute responsibility.

Exercise. Draw the boundary of CivicQueue twice: once narrowly around the web application and once around the complete appointment service. What becomes visible in the second drawing?

Worked solution. The wider boundary reveals staff routines, telephone fallback, identity services, notification providers, accessibility support, legal constraints, and the meaning of an appointment status. It also shows that technical success is only one condition for service success.

1.11 Further reading

For further reading, compare the sociotechnical perspective in **?**] with the information-systems success model in **?**], and explore the project-based learning literature if you are working collaboratively.

Chapter 2

Computational thinking, models, and simulations

2.1 A motivating problem

A city planner asks whether a change in housing allocation could reduce residential segregation. A programmer suggests a simulation in which agents move when too few neighbours are similar to them. The model is simple and often surprising: modest individual preferences can produce large-scale separation. But what exactly does the simulation show? It shows what follows from the rules of that model under its assumptions. It does not, by itself, estimate the effect of a real housing policy.

This chapter develops the habits needed to formulate problems computationally without confusing a model with reality. The same habits support web applications, algorithms, databases, and empirical research.

> **What you will learn.** You should be able to decompose a problem, identify abstractions and state variables, describe deterministic and stochastic transitions, implement a reproducible agent-based simulation, distinguish verification from validation, and discuss sensitivity and model limitations.

2.2 The five-minute explanation

Computational thinking is a family of problem-solving practices: decomposition, abstraction, representation, algorithm design, automation, and evaluation. It is not the claim that every problem should be reduced to code. It is a disciplined way to ask which parts of a problem can be represented by rules and which parts remain uncertain, normative, or outside the model.

> **In plain language.** A model is a deliberately simplified answer to the question "what features do we need to keep in order to study this behaviour?" Removing detail can make a model usable, but every removal is an assumption.

2.3 Decomposition and representation

Suppose CivicQueue must support rescheduling. Decompose it into smaller questions:

1. What counts as an appointment?
2. Which states can an appointment have?
3. Who may request a change?
4. Which time slots are available?
5. What happens when two requests arrive together?
6. Which systems and people must be notified?

This decomposition creates representations: a state machine, a data model, access-control rules, and a sequence of interactions. Different representations make different errors easy to see.

2.4 State and transition systems

Let X be a set of states and A a set of actions. A deterministic transition function is

$$T : X \times A \to X.$$

The notation says that, for a current state $x \in X$ and an action $a \in A$, the system produces one next state $T(x, a)$. If an action is not allowed, the transition is undefined or produces an error state. A stochastic transition can instead be written as a probability distribution $P(x' \mid x, a)$ over possible next states.

For an appointment, let $X = \{\text{requested}, \text{confirmed}, \text{cancelled}, \text{completed}\}$. An invariant might be that a cancelled appointment cannot become completed without a new appointment being created. The invariant is a claim about all allowed transitions, not merely an example trace.

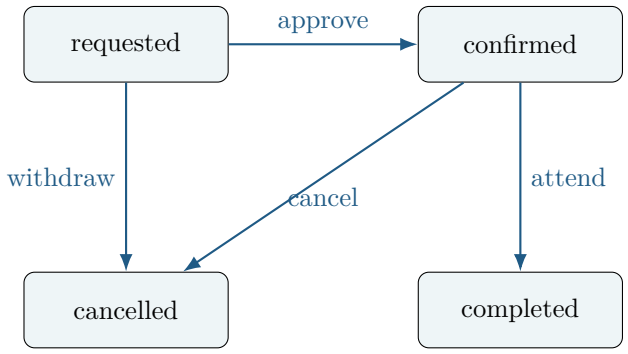

Figure 2.1: A small appointment state machine. An omitted transition is not automatically permitted.

2.5 Agent-based models

In an agent-based model, individual entities follow local rules while a simulation records the resulting system-level pattern. A minimal segregation model places agents on a grid. Each agent has a type, a neighbourhood, and a tolerance threshold. At each step, an unhappy agent moves to an empty location where its local condition is satisfied.

The model's main variables are:

- grid size and occupancy;
- agent types;
- neighbourhood definition;
- tolerance threshold;
- update order;
- random seed;
- stopping rule.

Changing any one can change the result. A random seed makes one run reproducible; it does not make a random process deterministic in the world.

The implementation in `examples/simulation/segregation.py` uses only the Python standard library. It returns a result object rather than printing from the model, which makes it easier to test and reuse.

Listing 2.1: A small transition rule from the simulation.

```
def is_happy(grid, row, col, tolerance):
    """Return whether the occupied cell satisfies its local rule."""
    agent = grid[row][col]
    if agent is None:
        return True
    neighbours = neighbours_of(grid, row, col)
    occupied = [item for item in neighbours if item is not None]
    if not occupied:
        return True
    similar = sum(item == agent for item in occupied)
    return similar / len(occupied) >= tolerance
```

The function does not claim that similarity is a complete description of social satisfaction. It is a formalisation chosen for a particular experiment.

2.6 Verification, validation, and sensitivity

Verification asks whether the implementation follows the specified model. Tests can check that an empty neighbourhood makes an agent happy, that no agent moves when all agents are happy, and that the same seed produces the same trace. *Validation* asks whether the model is an adequate representation for a purpose. That requires substantive argument and possibly comparison with data or expert knowledge.

For a parameter θ, a sensitivity analysis examines how an outcome Y changes when θ changes. A simple finite-difference measure is

$$\Delta Y = Y(\theta + \delta) - Y(\theta),$$

where δ is a specified change. The quantity is meaningful only when the outcome, parameter range, randomisation procedure, and comparison are stated. If results change dramatically under plausible values, conclusions should be conditional rather than absolute.

> **Caution.** A simulation can be internally correct and externally irrelevant. A realistic-looking animation is not evidence of validation. State the target phenomenon, the purpose of the model, and the assumptions that connect the model to that phenomenon.

2.7 Other computational models

Conway's Game of Life uses a local rule on a grid and demonstrates emergence: global patterns arise from repeated local transitions. Flocking models represent agents with rules for separation, alignment, and cohesion. Epidemic models represent contact and transmission. Network diffusion represents entities as nodes and relations as edges. Schelling-style models represent local preferences and movement. These examples teach formalisation and emergent behaviour, but they should not be presented as direct explanations of human society.

2.8 Parameters and experimental design

An experiment is easier to interpret when it changes one planned factor at a time or uses a documented factorial design. Record:

1. the research or modelling question;
2. the outcome measure and its units;
3. parameter values and ranges;
4. number of replications;
5. random seeds;
6. software version and environment;
7. analysis procedure and stopping rules.

Reproducibility is not the same as truth. It is a condition for checking the reasoning that produced a result.

2.9 Project application

For a computational-thinking project, begin with a problem from a domain you understand. Define the target phenomenon before selecting a technique. Include a model diagram, a table of variables, pseudocode, implementation tests, at least one sensitivity analysis, and a section called "What this model cannot establish." When presenting the work, be prepared to change a parameter or rule and explain why the interpretation changes.

2.10 Summary and key terms

Computational thinking makes choices about decomposition, representation, and automation explicit. State-transition models support reasoning about software. Agent-based models can reveal emergence from local rules. Verification concerns implementation against a specification; validation concerns adequacy for a purpose. Random seeds, sensitivity analysis, and documented parameters support reproducibility and honest interpretation.

> **Key terms.** computational thinking; abstraction; representation; state; transition; invariant; deterministic; stochastic; agent-based model; emergence; verification; validation; sensitivity analysis; reproducibility.

2.11 Exercises and worked solutions

Exercise. Write an invariant for CivicQueue's appointment states and give one invalid transition.

Worked solution. One invariant is that an appointment marked completed must have previously been confirmed. An invalid transition is cancelled → completed. A legitimate new booking after cancellation should create a new appointment identifier rather than silently changing the old record.

Exercise. Why does a random seed help a simulation study? What does it not do?

Worked solution. It makes a particular pseudo-random sequence reproducible, allowing a reviewer to rerun the same case. It does not remove uncertainty, guarantee that the seed is representative, or validate the model.

Exercise. Design a sensitivity analysis for the segregation model.

Worked solution. Choose a tolerance range, for example 0.3 to 0.8, specify grid size and occupancy, run multiple seeds at each value, and report the mean and variation of a pre-defined segregation measure. Do not change the stopping rule or outcome definition between parameter values without documenting it.

2.12 Further reading

Computational thinking connects algorithm design, abstraction, automation, data representation, and agent-based formalisation. For algorithmic foundations, see **?**]; for the relation between models and computation, see **?**].

Chapter 3

Programming from first principles

3.1 A motivating problem

Suppose a staff member asks for a script that lists appointments occurring tomorrow. A first attempt might appear to work on a small sample, but a real system must also handle time zones, missing data, invalid dates, duplicate records, permissions, and tests. Programming is the activity of expressing a procedure so that a computer can execute it. Good programming also requires making assumptions, boundaries, and failure behaviour explicit.

> **What you will learn.** You should be able to explain how source code reaches execution, use Python values and control flow correctly, design functions with clear contracts, handle errors, work with files and structured data, and write tests that expose assumptions.

3.2 The five-minute explanation

A program is a sequence of instructions interpreted by a runtime or translated by a compiler into a lower-level representation. Python source is parsed into an internal representation and executed by the Python runtime. The exact implementation varies, but the enduring ideas are source text, syntax, semantics, state, control flow, and effects.

> **In plain language.** A variable is not a labelled box containing a string of characters. In Python it is a name bound to an object. Assignment changes a binding; mutation changes an object. Keeping those ideas separate prevents many bugs.

3.3 Values, names, and types

Python values include integers, floating-point numbers, booleans, strings, lists, dictionaries, and user-defined objects. A type describes the operations and representation available for values of a kind. Python checks many type constraints at runtime; static type checkers can detect additional inconsistencies before execution.

Listing 3.1: Bindings, mutation, and a type-annotated function.

```
def add_fee(amount: float, fee: float) -> float:
    if amount < 0 or fee < 0:
        raise ValueError("amounts must be non-negative")
    return amount + fee

first = ["requested"]
second = first
second.append("confirmed")
assert first == ["requested", "confirmed"]
```

The two names refer to the same list object. If independent lists are needed, create a copy with `first.copy()`. An assertion expresses an assumption that should remain true at that point in the program.

3.4 Expressions and control flow

An expression produces a value. A statement performs an action such as assignment, branching, looping, or returning. A conditional is a choice between paths. A loop repeats a block while a condition holds or for each element of an iterable. The important question is not how many keywords a language has, but which control-flow graph the program creates.

Listing 3.2: Filtering appointments with an explicit predicate.

```
def confirmed_for_day(appointments: list[dict], day: str) -> list[dict]:
    """Return confirmed appointments whose ISO date equals day."""
    return [
        appointment
        for appointment in appointments
        if appointment.get("status") == "confirmed"
        and appointment.get("date") == day
    ]
```

The function uses a list comprehension, but the same logic can be written as a loop. Choose the form that makes the invariant and failure behaviour clearest to the team.

3.5 Functions as contracts

A function should have a small purpose, named inputs, a documented output, and an explicit error policy. A contract can be described as:

$$\{P\}\ C\ \{Q\},$$

where P is a precondition, C is the command or function, and Q is a postcondition. For `add_fee`, P is that both inputs are non-negative numbers; Q is that the returned value equals their sum and is non-negative. A contract does not make a function correct automatically. It makes the claim inspectable and testable.

3.6 Input, output, and files

External input is untrusted and may be absent, malformed, stale, or malicious. Parse it at the boundary, validate it, and convert it to an internal representation. JSON is a data

interchange format, not a security boundary. Never treat data from a request as executable code.

Listing 3.3: Safe JSON loading at a file boundary.

```
import json
from pathlib import Path

def load_records(path: Path) -> list[dict]:
    with path.open(encoding="utf-8") as handle:
        data = json.load(handle)
    if not isinstance(data, list):
        raise ValueError("expected a JSON array")
    if not all(isinstance(item, dict) for item in data):
        raise ValueError("every record must be a JSON object")
    return data
```

Use context managers so files close even if parsing raises an exception. Validate the shape before using fields. In a production service, add limits on file size and record count.

3.7 Errors, exceptions, and debugging

An error is a mismatch between the program's assumptions and the current situation. Exceptions are one mechanism for transferring control to an error handler. Catch exceptions at a boundary where recovery is possible; do not catch every exception and silently continue. A useful debugging report records the smallest input that reproduces the problem, expected behaviour, actual behaviour, and the first violated assumption.

3.8 Testing and executable examples

The examples directory contains tests alongside the code. A unit test checks a small behaviour in isolation; an integration test checks interactions among components; an end-to-end test follows a user-visible path. Tests are executable claims. They do not remove the need for inspection, threat modelling, accessibility evaluation, or exploratory testing.

Listing 3.4: A small test for the appointment filter.

```
def test_confirmed_for_day_ignores_other_statuses():
    rows = [
        {"status": "confirmed", "date": "2026-09-01"},
        {"status": "cancelled", "date": "2026-09-01"},
        {"status": "confirmed", "date": "2026-09-02"},
    ]
    assert confirmed_for_day(rows, "2026-09-01") == [rows[0]]
```

The test is precise about status and date. It does not test time zones, malformed records, or permissions; those belong in further tests.

3.9 Web foundations

HTML represents document structure; CSS describes presentation; JavaScript can manipulate the Document Object Model and respond to browser events. HTTP carries requests and responses between clients and servers. A browser event may trigger a request, which invokes server logic, changes persistence, returns a response, and produces visible feedback. Chapter 8 develops the full path.

3.10 Code quality and security

Readable names, small functions, consistent formatting, dependency control, and version history reduce the cost of reasoning. Security is a property of the whole system: input validation, authentication, authorisation, secrets management, logging, dependency updates, and safe defaults must be designed together. OWASP's Top 10 is a useful awareness list, not a complete security specification [**?**].

3.11 Project application

Start a project with a small vertical slice: one user action, one domain rule, one persistence operation, one visible result, and one test. This makes integration risks visible early. When documenting the work, show the requirements traceability from user need to function to test. When presenting it, explain which code is deliberately simple and what would change for production.

3.12 Summary and key terms

Programming combines representation, control flow, state, functions, effects, and error handling. Names refer to values; mutation and rebinding differ. Contracts express assumptions. Tests provide executable evidence about specified behaviours. Secure and maintainable code requires boundary validation, clear ownership, and feedback from execution.

> **Key terms.** source code; runtime; value; object; binding; type; expression; statement; control flow; function; scope; precondition; postcondition; invariant; exception; test; module; dependency; API.

3.13 Exercises and worked solutions

Exercise. What is the difference between `second = first` and `second = first.copy()` when `first` is a list?

Worked solution. The first assignment creates a second name for the same list, so mutation through either name is visible through both. `copy()` creates a new list with the same current elements, so appending to one list does not append to the other. Nested mutable objects require a deeper copy if independent nested state is needed.

Exercise. Write a contract for a function that reserves an appointment slot.

Worked solution. A possible precondition is that the slot exists, is in the future, and the caller is authorised. The postcondition is that exactly one reservation is created and the slot is no longer available, or that the operation returns a well-defined conflict without changing state. Concurrency means the postcondition must hold even when two requests arrive together.

Exercise. Name three tests that the appointment filter still needs beyond the example.

Worked solution. Tests could cover an empty input, a missing `status` field, a malformed date, duplicate records, and a date with no confirmed appointments. The correct selection depends on the contract and threat model; a test is useful only when its expected behaviour is justified.

3.14 Further reading

Good programming practice connects computational thinking, data structures, functions, web basics, testing, and the evaluation of programming solutions. For a gentle introduction, see **?**]; the Python documentation is the reference for language behaviour [**?**].

Chapter 4

Algorithms, data structures, and formal reasoning

4.1 A motivating problem

CivicQueue may store 100 appointments today and ten million historical records next year. A search that feels instant on a laptop can become unusable at scale. Choosing an algorithm and data structure is therefore a design decision about time, memory, correctness, and future change.

> **What you will learn.** You should be able to use sets, relations, functions, sequences, trees, and graphs; explain invariants and induction; implement common data structures; analyse time and space asymptotically; and distinguish asymptotic reasoning from measured performance.

4.2 Sets, relations, and functions

A set is a collection of distinct elements. If $A = \{1, 2, 3\}$, then $2 \in A$ means 2 is a member. A relation R between sets A and B is a set of ordered pairs from $A \times B$. A function $f : A \to B$ is a relation with exactly one output in B for each input in A.

These ideas appear in systems. Let U be users and P be permissions. A relation `can_access` $\subseteq U \times P$ records grants. It may not be a function: one user can have several permissions, and several users can share one permission. Confusing a relation with a function causes bad data models and incorrect code.

4.3 Data structures as representations

An array provides indexed access. A stack supports last-in, first-out operations. A queue supports first-in, first-out operations. A hash table maps keys to buckets using a hash function. A tree represents hierarchy; a graph represents arbitrary relationships.

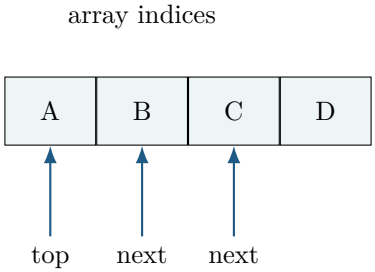

Figure 4.1: A sequence can support several abstract data types, but the operations and invariants differ.

An abstract data type specifies allowed operations and their meaning without fixing an implementation. A queue can be implemented by a list, a linked structure, or a concurrent service. The abstraction helps clients depend on behaviour rather than representation.

4.4 Correctness through invariants

An invariant is a proposition that is true before and after every operation that preserves the data structure. For a queue, an invariant might be that the front element is the oldest element not yet removed. To reason about a loop, identify:

1. the invariant before the first iteration;
2. why one iteration preserves it;
3. what the invariant implies when the loop stops.

This is a practical form of proof. It can be applied to a cursor in a database query, a parser, or a simulation update.

4.5 Induction and recursion

Mathematical induction proves a statement for all natural numbers. First prove the base case. Then assume it holds for an arbitrary n and prove it holds for $n + 1$. Recursive programs use a related structure: a base case and a recursive case that reduces the problem to a smaller instance. A recursive function without a reachable base case does not terminate.

For the sum $1 + 2 + \cdots + n$, define $S(0) = 0$ and $S(n) = n + S(n - 1)$. The recurrence makes the reasoning explicit. An iterative implementation may be preferable in Python when recursion depth is large.

4.6 Searching and sorting

Linear search checks elements one by one. If the target is equally likely to be anywhere, its average number of comparisons is approximately $(n + 1)/2$ when it exists and n when it does not. Binary search repeatedly halves a sorted search interval. Its logarithmic behaviour depends on the precondition that the data is sorted and that the index operation is efficient.

Sorting algorithms illustrate trade-offs. Insertion sort is simple and useful for small or nearly sorted inputs. Merge sort offers $O(n \log n)$ worst-case time and uses extra space. Quicksort is often fast in practice but can have $O(n^2)$ worst-case time unless the pivot strategy and input assumptions are controlled. A library implementation may be preferable because it has been engineered and tested across cases.

4.7 Asymptotic notation

We write $f(n) \in O(g(n))$ if there are constants $c > 0$ and n_0 such that

$$0 \le f(n) \le c\, g(n) \quad \text{for every } n \ge n_0.$$

The variable n measures input size. The notation describes an eventual upper bound up to constant factors. It ignores constant costs, hardware, cache behaviour, interpreter overhead, and distribution of inputs. Therefore $O(n)$ is not automatically faster than $O(n \log n)$ for small n, and the label does not predict latency without a cost model.

Space complexity applies the same idea to additional memory. A streaming algorithm may use $O(1)$ extra space while taking $O(n)$ time. A hash table may offer expected $O(1)$ lookup under assumptions about hashing and load, while requiring $O(n)$ storage.

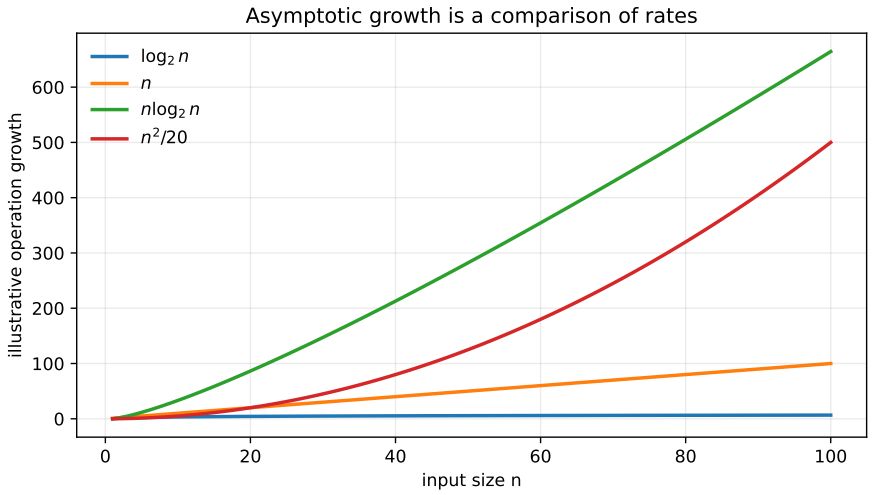

Figure 4.2: Illustrative growth curves generated by `figures/complexity_plot.py`. The plot is a teaching aid, not a benchmark.

4.8 Graphs and networks

A graph $G = (V, E)$ consists of vertices V and edges E. Edges may be directed or undirected and may carry weights. In CivicQueue, a graph could represent dependencies among services, escalation paths, or relations among organisations. Breadth-first search explores by distance in an unweighted graph; Dijkstra's algorithm solves shortest paths when edge weights are non-negative. The representation and query determine which algorithm is appropriate.

4.9 Measured performance

Asymptotic analysis and measurement answer different questions. A benchmark should state hardware, software version, input generation, warm-up, repetitions, timing method, and variability. Avoid measuring a toy implementation and generalising to a production system. Profile before optimising; a theoretically elegant algorithm cannot compensate for a database query that returns unnecessary rows or a network call made inside a loop.

4.10 Project application

For any non-trivial feature, record the data structure, operation costs, input-size assumptions, and correctness invariant. When documenting the work, justify the choice in relation to the problem rather than listing Big-O labels. When presenting it, explain a failure case: an unsorted input to binary search, a hash collision, a disconnected graph, or an unexpectedly large recursion depth.

4.11 Summary and key terms

Formal structures clarify what data means. Algorithms are procedures with preconditions, operations, and correctness claims. Invariants and induction provide reusable reasoning tools. Asymptotic notation describes growth under explicit assumptions; measurement is still needed for engineering decisions.

> **Key terms.** set; relation; function; sequence; abstract data type; array; stack; queue; hash table; tree; graph; invariant; induction; recursion; sorting; searching; asymptotic notation; scalability.

4.12 Exercises and worked solutions

Exercise. A list contains 1,000,000 user identifiers. When is a set a better representation than a list?

Worked solution. If the main operation is membership testing and order is not required, a set usually offers expected constant-time membership rather than linear scanning. It uses additional memory and depends on hash behaviour. If stable ordering or duplicate values matter, another representation may be more suitable.

Exercise. State the loop invariant for a linear search that has examined positions 0 through $i - 1$.

Worked solution. The target is not present in positions 0 through $i - 1$, unless the algorithm has already returned. Initially the range is empty, so the invariant holds. Each iteration checks the next position and preserves the claim if the target is not found. When $i = n$, the invariant establishes that the target is absent.

Exercise. Why is binary search not valid on an arbitrary list?

Worked solution. Binary search discards half of the remaining range because sorted order guarantees that all values on one side are too small or too large. Without sorted order, discarding a side can discard the target. The algorithm's logarithmic bound depends on this precondition.

4.13 Further reading

See **?**] for algorithms and correctness, and **?**] for a deeper historical treatment of representation and analysis.

Part II

Engineering systems

Chapter 5

Software engineering: from uncertainty to dependable systems

5.1 A motivating problem

A team can write ten thousand lines of code and still fail to deliver a useful system. Requirements may conflict, the deployment environment may differ from the developer's laptop, and a small change to an appointment rule may break reporting. Software engineering exists because software is a socio-technical product made under constraints, not because programmers need a larger vocabulary for writing code.

> **What you will learn.** You should be able to distinguish programming from software engineering; elicit and specify requirements; connect requirements to design, code, and tests; compare lifecycle approaches; reason about quality attributes and technical debt; and use version control and review as engineering practices.

5.2 The five-minute explanation

Programming asks how to make a particular computation work. Software engineering asks how a team can repeatedly produce, deploy, operate, and change software while preserving agreed properties. Those properties include correctness, reliability, security, performance, usability, accessibility, maintainability, and compliance. They compete for resources and sometimes conflict.

A requirement is not merely a feature request. It is a statement about a needed capability or constraint in a context. A functional requirement says what the system should do. A non-functional requirement describes a quality, constraint, or condition under which it must do it. A vague requirement such as "the system should be fast" is not testable until response conditions and a threshold are specified.

5.3 The problem is part of the engineering work

CivicQueue begins with observations, interviews, existing documents, and operational data. A team should record the source and status of each statement:

- **Need:** citizens need a way to request a change without a telephone call.
- **Requirement:** an authenticated citizen can request a new slot from the available slots shown for the relevant service.
- **Acceptance criterion:** given a confirmed appointment and an available future slot, when the citizen confirms the change, the old slot is released only if the new reservation succeeds.
- **Assumption:** the external calendar service has the same time-zone definition as CivicQueue.
- **Risk:** a simultaneous request could reserve the same slot twice.

Traceability links these statements to design decisions, implementation, and tests. It does not make requirements true; it makes missing reasoning easier to discover.

5.4 Lifecycle models and their limits

A lifecycle model is a coordination model, not a law of nature. A plan-driven process can be appropriate when requirements are stable, change is expensive, and verification must be documented. Incremental development delivers slices that create feedback. Agile methods emphasise working increments, collaboration, and response to change [**?**]. A spiral model organises work around risk and learning [**?**].

The contrast between waterfall and agile is often overstated. Real projects mix discovery, design, implementation, testing, procurement, compliance, and operation. The useful question is which uncertainty should be reduced before commitment and which should be explored through an increment.

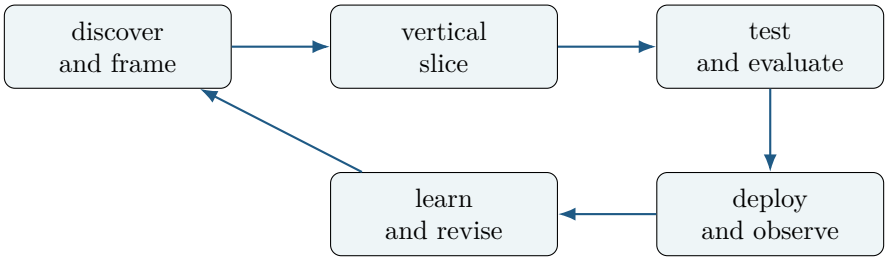

Figure 5.1: An incremental engineering loop that keeps discovery, delivery, and learning connected.

5.5 Quality attributes and trade-offs

A quality attribute becomes useful when it has a context, a measure, and a threshold. For CivicQueue:

- availability: the service is available during published booking hours;
- confidentiality: citizens cannot view another person's case;
- response time: the booking confirmation appears within a stated percentile under a stated load;

- accessibility: core tasks can be completed with keyboard navigation and assistive technology;
- maintainability: domain rules can be changed without editing presentation code.

These are not independent. Aggressive caching may improve speed but expose stale or private data. Strong verification may slow release but reduce risk. The team should document the trade-off rather than claim that all qualities are maximised.

5.6 Version control, review, and integration

Git records snapshots of a project and supports branching, comparison, and collaboration. A branch is a coordination mechanism, not a substitute for communication. A useful change has a small scope, a clear message, tests, and a review that examines behaviour and risk. Merging code is easy compared with integrating assumptions.

Continuous integration runs repeatable checks when changes are proposed. A minimal pipeline may format code, run unit tests, run integration tests, build the application, and produce an artefact. Continuous delivery adds a controlled path to deployment. Neither practice guarantees that a requirement is correct or that the system is usable.

5.7 Technical debt and maintenance

Technical debt is not simply bad code. It is a present choice that creates future cost, risk, or reduced option value. Some debt is deliberate and documented; some is accidental. A team should record why it exists, what interest it incurs, and when it should be repaid. Refactoring changes internal structure while preserving observable behaviour. It is safer when tests describe the behaviour that must remain stable.

5.8 Professional responsibility

A software engineer is accountable for communicating uncertainty and foreseeable harm. Security, privacy, accessibility, and inclusive design should appear in requirements and tests, not as a final polish step. A team should have an escalation path for unsafe requests, missing consent, data leakage, and conflicts between delivery pressure and professional standards.

5.9 Project application

A strong project report can include:

1. a problem formulation and stakeholder evidence;
2. a requirements table with source, priority, and acceptance criteria;
3. a decision log for architecture and scope;
4. a traceability matrix from requirement to test;
5. a delivery history showing iterations and feedback;

6. a quality and risk assessment;
7. a reflection on group work and technical debt.

When presenting the work, do not turn the process into a success story. Explain a decision that was revised, an assumption that failed, and what evidence caused the change.

5.10 Summary and key terms

Software engineering coordinates people, models, code, tests, deployment, and maintenance under constraints. Requirements should be traceable and testable. Lifecycle models organise uncertainty but do not remove it. Quality attributes require context and trade-offs. Version control, review, integration, and technical-debt management are mechanisms for preserving reasoning across time.

> **Key terms.** software engineering; requirement; acceptance criterion; traceability; lifecycle; incremental development; agile; quality attribute; reliability; technical debt; refactoring; version control; continuous integration; deployment.

5.11 Exercises and worked solutions

Exercise. Turn "the booking process should be user-friendly" into two testable requirements.

Worked solution. One requirement could be that a first-time user can identify the rescheduling action from the appointment view without help in a defined usability test. Another could be that the rescheduling form can be completed using keyboard navigation and has a programmatically associated error message for each invalid field. The thresholds and study procedure must still be specified.

Exercise. What is the difference between a feature and an acceptance criterion?

Worked solution. A feature names a capability, such as rescheduling. An acceptance criterion states an observable condition under which a particular behaviour is considered acceptable, including relevant inputs, outputs, and failure handling.

Exercise. Why is a small vertical slice useful before implementing the whole system?

Worked solution. It exposes integration assumptions among interface, domain logic, persistence, and deployment. A broad component can look complete while the system cannot perform one user-visible task end to end.

5.12 Further reading

Software development connects programming to requirements, demonstrable running systems, and explanations of how programs solve problems. Compare the Agile Manifesto with the Scrum Guide [**?**], and treat both as historical and practical sources rather than universal laws.

Chapter 6

Object-oriented and domain design

6.1 A motivating problem

CivicQueue contains appointments, citizens, service types, slots, notifications, and staff actions. If the system stores everything as unrelated dictionaries, rules become scattered and inconsistent. If it creates a large hierarchy of classes, a small policy change can require changes across the entire program. Object-oriented design is a set of modelling and programming techniques for managing such complexity, not a command to turn every noun into a class.

> **What you will learn.** You should be able to explain objects, identity, state, behaviour, encapsulation, interfaces, composition, inheritance, polymorphism, cohesion, coupling, dependency inversion, immutability, and testability; compare object-oriented design with procedural, functional, data-oriented, and component-based approaches; and refactor a domain model.

6.2 The five-minute explanation

An object combines identity, state, and behaviour. A class is one way to define a family of objects. Encapsulation means that clients depend on a controlled interface rather than directly changing internal representation. An invariant is a condition the object promises to preserve.

For an appointment, the rule “a cancelled appointment cannot be completed” belongs close to the state it protects. This does not mean the object should contain database access, HTML rendering, and email delivery. Good design separates responsibilities while keeping domain rules explicit.

6.3 A small domain model

The following example uses Python dataclasses. It is intentionally incomplete; a production model would need time zones, identifiers, authorisation, persistence, and concurrency control.

Listing 6.1: A domain object with an explicit state invariant.

```
from dataclasses import dataclass
```

```
from enum import Enum

class Status(Enum):
    REQUESTED = "requested"
    CONFIRMED = "confirmed"
    CANCELLED = "cancelled"
    COMPLETED = "completed"

@dataclass
class Appointment:
    identifier: int
    status: Status = Status.REQUESTED

    def confirm(self) -> None:
        if self.status is not Status.REQUESTED:
            raise ValueError("only requested appointments can be confirmed")
        self.status = Status.CONFIRMED

    def cancel(self) -> None:
        if self.status in {Status.CANCELLED, Status.COMPLETED}:
            raise ValueError("appointment cannot be cancelled")
        self.status = Status.CANCELLED

    def complete(self) -> None:
        if self.status is not Status.CONFIRMED:
            raise ValueError("only confirmed appointments can be completed")
        self.status = Status.COMPLETED
```

The methods define allowed transitions. The class does not know how the object is stored or displayed. This separation makes unit tests cheap and reduces coupling.

6.4 Composition, inheritance, and polymorphism

Composition builds an object from collaborating objects. Inheritance creates a subtype relation and can support substitutability, but it also couples subclasses to superclass decisions. Prefer composition when the relationship is “has a” or when behaviour should be selected at runtime. Use inheritance when a stable substitutable abstraction is genuinely present.

Polymorphism means that code can use a common interface while the concrete behaviour varies. In Python, protocols and duck typing can express this without a class hierarchy. A notification service might accept any object with a `send` method. The abstraction should be defined by the responsibility the caller needs, not by a taxonomy of vendors.

6.5 SOLID principles as questions

SOLID principles are heuristics, not proof rules:

Single responsibility Which reason for change does this module own?

Open–closed Can a new variant be added without editing stable code?

Liskov substitution Does a subtype preserve the promises clients rely on?

Interface segregation Are clients forced to depend on operations they do not use?

Dependency inversion Does high-level policy depend on abstractions rather than infrastructure details?

Applying a principle mechanically can produce needless indirection. A small program may be clearer with a function than with five interfaces.

6.6 Cohesion, coupling, and immutability

Cohesion asks whether the responsibilities in a module belong together. Coupling asks how much one module knows about another. Immutability reduces the number of possible state histories. A value object such as a `SlotId` or a date range can be immutable and validated at construction. Mutable aggregate objects require stronger invariants and careful ownership.

6.7 Patterns and their forces

A design pattern describes a recurring problem, the forces that shape it, a structure, and consequences. The Repository pattern can separate domain code from persistence; Dependency Injection can make infrastructure replaceable in tests; Adapter can translate an external API; Strategy can select an algorithm; Observer can notify dependants about an event. MVC separates concerns in interactive applications, but the boundaries depend on the framework and can become confused.

The common misuse is pattern naming without problem analysis. Ask what change or force the pattern addresses, what complexity it introduces, and whether a simpler function or module would be better. The original catalogue remains useful as a language for design discussion [**?**].

6.8 Alternatives to object orientation

Procedural decomposition can be clearer for transformations. Functional programming emphasises pure functions and immutable data. Data-oriented design organises around memory layout and operations over data. Component-based design composes independently deployable or replaceable parts. A realistic system often combines all four. The criterion is clarity of invariants and change boundaries, not ideological consistency.

6.9 Testing and refactoring

Test domain objects through public behaviour. Test an invariant's boundary cases, not just the happy path. Refactor after a test suite gives a safety net. If a class requires a database, clock, network, and global configuration to construct, the design is telling you that responsibilities are mixed or dependencies are hidden.

6.10 Project application

When documenting the work, include a domain model and explain which concepts are entities, value objects, services, or external systems. Show one design alternative you rejected and the force that made it less suitable. When presenting it, explain why a domain rule lives where it does and how the design would change if a second notification channel were added.

6.11 Summary and key terms

Objects protect state and expose behaviour through interfaces. Composition is often safer than inheritance for reuse. Polymorphism supports substitutable variation. SOLID principles are questions about change, substitution, and dependency, not automatic templates. Patterns are named responses to forces. Procedural, functional, data-oriented, and component-based designs remain legitimate alternatives.

> **Key terms.** object; identity; state; behaviour; class; instance; encapsulation; abstraction; interface; composition; inheritance; polymorphism; dynamic dispatch; cohesion; coupling; immutability; dependency inversion; design pattern.

6.12 Exercises and worked solutions

Exercise. Why should an appointment object reject an invalid transition rather than let any caller assign its status?

Worked solution. If callers can assign arbitrary status values, the invariant is distributed across every caller and can be violated by omission. A method or validated state transition centralises the rule. The object still needs integration tests and authorisation at a higher level; encapsulation does not solve every concern.

Exercise. Give one reason to prefer composition over inheritance for notifications.

Worked solution. A service may need to send through email, SMS, or a queue depending on configuration. Composing an appointment service with a notification interface allows the strategy to change without making `EmailNotification` and `SmsNotification` subclasses of an artificial common object.

Exercise. When can a design pattern make a project worse?

Worked solution. It can add indirection, obscure a simple control flow, create more classes to maintain, or imply a fixed structure when the forces are not present. The pattern should earn its complexity by addressing a real change or coordination problem.

6.13 Further reading

Use access, encapsulation, resource management, and security as questions for analysing a design. Compare the patterns catalogue [**?**] with the enterprise patterns in **?**].

Chapter 7

Systems analysis and modelling

7.1 A motivating problem

A team says that CivicQueue needs “a calendar integration”. One person imagines a public booking calendar; another imagines a staff scheduling service; a third imagines a synchronisation job. The sentence is too compressed to coordinate design. A model externalises the assumptions so that people can inspect, question, and revise them.

> **What you will learn.** You should be able to use models to analyse and communicate systems; define boundaries, actors, responsibilities, events, interfaces, and contracts; choose appropriate UML diagrams; explain design patterns as responses to forces; and assess model–code consistency.

7.2 The five-minute explanation

A model is a selective representation made for a purpose. A use-case model helps discuss goals and actors. A class model describes concepts and relationships. A sequence model describes an interaction over time. A state machine describes legal transitions. A component model describes replaceable units and dependencies. No diagram is the system itself.

> **In plain language.** A diagram is valuable when it lets a team notice, remember, or negotiate something that would otherwise remain hidden. It is harmful when its symbols create a false sense of precision.

7.3 Problem and application domains

The problem domain contains the situation the system is meant to address: appointments, citizens, staff work, and rules. The application domain contains the technical artefact and its interfaces. A good model relates the two without assuming that a software class is identical to a real-world institution.

Define the system boundary before drawing. List actors by the goals they pursue, not merely by the screens they use. An external identity provider may be an actor in a system interaction even when no human directly sees it.

7.4 Use cases and requirements

A use case describes a goal-oriented interaction. “Reschedule appointment” is a goal; “click button” is an interaction step. A use-case narrative should include preconditions, trigger, main success scenario, alternatives, failure conditions, and postconditions.

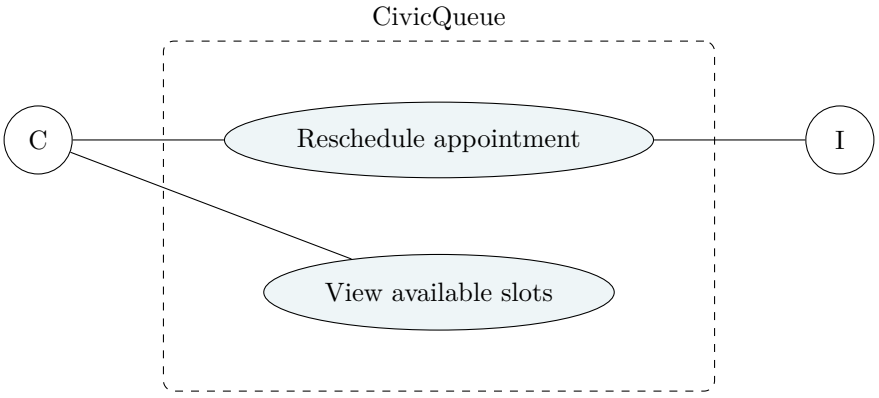

Figure 7.1: A use-case boundary with a citizen actor and an external identity service.

The abbreviations C and I are explained in the caption or surrounding text. Avoid relying on a visual legend alone.

7.5 Class and domain models

A class diagram can show entities, value objects, associations, multiplicities, and responsibilities. It should distinguish an appointment from an appointment slot: one is a booked occurrence; the other is an available capacity at a time and service location. If a rescheduling operation creates a new appointment, the model should make that identity decision visible.

A domain model is not a database schema. It may contain concepts that are derived, transient, or intentionally absent from persistence. Conversely, a database may contain technical fields such as audit timestamps and indexes that do not belong in the domain vocabulary.

7.6 Behavioural models

A sequence diagram shows messages between participants:

1. the citizen submits a rescheduling request;
2. the interface sends an HTTP request;
3. the application authorises the action;
4. the domain service checks the state transition;
5. the repository performs a transaction;
6. the response reports success or a conflict;

7. the interface gives feedback.

A state machine models legal histories. An activity diagram models control and responsibility. Use the smallest model that answers the question being asked.

7.7 Contracts

A contract specifies an interface's preconditions, postconditions, and invariants. For a slot-reservation operation:

- precondition: the caller is authorised and the slot is available;
- postcondition on success: exactly one reservation owns the slot;
- postcondition on conflict: the slot ownership is unchanged;
- invariant: one slot cannot have two active reservations.

Contracts expose concurrency requirements that a class diagram alone may hide.

7.8 Architecture and patterns

At a high level, CivicQueue can be separated into presentation, application services, domain logic, persistence, and external adapters. MVC can help organise a web interface. Repository hides persistence details. Adapter isolates a calendar provider. Dependency Injection makes the application service testable. Observer may model notifications, but asynchronous delivery introduces retries and duplicate-event concerns.

Patterns are not decorations. For each pattern, state the problem, forces, structure, consequences, alternatives, and misuse. A repository is not automatically better than a direct query; it is useful when the persistence boundary is meaningful and stable.

7.9 Model–code consistency

Models decay when code changes without updating them, but freezing every diagram can slow development. Keep diagrams that support decisions, onboarding, safety, or communication. Link them to code modules and tests. Delete a diagram when it no longer represents a decision that anyone uses.

7.10 Project application

Choose one diagram per question. Include a system boundary early, a domain model before implementation, and a sequence or state model for the most failure-prone interaction. When documenting the work, explain abstraction choices and inconsistencies. When presenting it, be prepared to redraw the model when someone changes an actor, permission, or failure condition.

7.11 Summary and key terms

Modelling is a way of thinking and coordinating, not diagram production. Different diagrams answer different questions. Boundaries, actors, responsibilities, events, interfaces, contracts, and invariants connect problem analysis to implementation. Patterns are justified by forces and consequences. Models should be maintained selectively and honestly.

> **Key terms.** model; system boundary; actor; use case; domain model; UML; class diagram; sequence diagram; state machine; activity; component; deployment; interface; contract; precondition; postcondition; design pattern; architecture.

7.12 Exercises and worked solutions

Exercise. Why is "calendar integration" a poor requirement? Rewrite it as a use-case goal and one acceptance criterion.

Worked solution. It names a technical solution without a user goal or boundary. A goal is "a staff member can publish available appointment slots from the authoritative schedule". An acceptance criterion could state that a published slot is shown with the correct service, time zone, and availability, and that a failed synchronisation leaves the previous published state visible with an error recorded.

Exercise. Which diagram is most useful for showing that a cancelled appointment cannot be completed?

Worked solution. A state-machine diagram directly represents legal transitions. A class diagram may show a status attribute, and a sequence diagram may show one scenario, but neither alone states all allowed histories as clearly.

Exercise. What makes a diagram misleading?

Worked solution. Ambiguous names, hidden boundaries, missing failure paths, inconsistent notation, false precision, and an outdated relationship to code can all mislead. A diagram should state its purpose and scope.

7.13 Further reading

The UML specification provides a standard notation [**?**]. This book treats modelling as a means of reasoning rather than a compliance exercise.

Chapter 8

Web and full-stack application development

8.1 A motivating problem

A citizen clicks “request new time”. The browser sends a request through a network. The server authenticates the user, checks authorisation, validates the input, applies a domain rule, writes data, and returns a response. The browser renders feedback. If any step is misunderstood, the prototype may work only on the developer’s machine.

> **What you will learn.** You should be able to trace a request through a full-stack system; explain HTTP, REST-style resource design, JSON, client and server responsibilities, authentication and authorisation; design validation and error handling; and discuss configuration, deployment, monitoring, and architectural trade-offs.

8.2 The five-minute explanation

A full-stack application is a set of cooperating layers:

1. the client presents information and captures interaction;
2. the network transports a request and response;
3. the server applies application and domain logic;
4. persistence stores and retrieves durable state;
5. external services provide identity, notifications, or other capabilities;
6. operation practices keep the system observable and changeable.

The layers are not physical boxes in every deployment. They are responsibilities that should be identifiable.

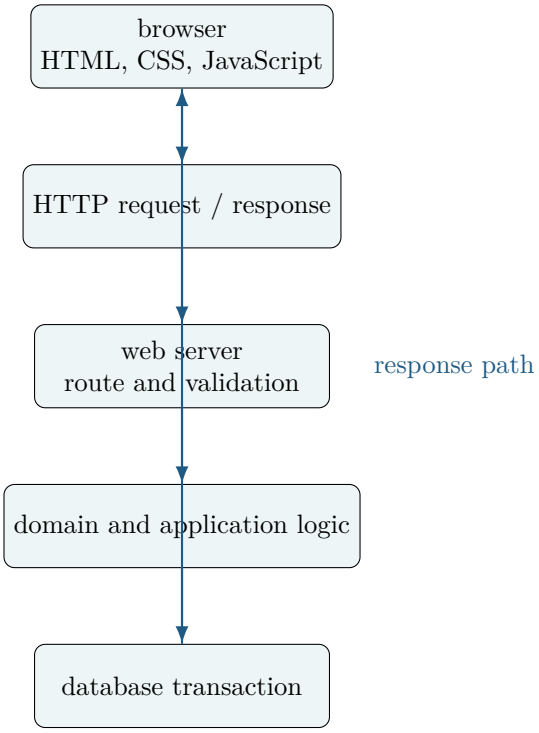

Figure 8.1: The path of a CivicQueue request. Every arrow is a possible failure boundary.

8.3 HTTP and resource design

HTTP has methods, status codes, headers, and a message body. A `GET` should retrieve a representation without changing server state; a `POST` commonly requests creation or an action; `PUT` replaces a resource representation; `PATCH` partially modifies it; `DELETE` requests removal. These are conventions with semantics, not magic names.

A REST-style API exposes resources and representations. For CivicQueue, a client might request `GET /api/appointments/42` or `POST /api/appointments/42/reschedule`. The second path is an action endpoint; a different design could model a rescheduling request as a new resource. The choice should reflect domain meaning and idempotency.

8.4 Frontend state and feedback

The browser has state: the current route, form values, loading status, errors, and cached data. Do not show success before the server confirms success. Disable or de-duplicate a submission only when the interaction design and server semantics support it. Accessible feedback must be perceivable by people who do not see a visual colour change.

8.5 Backend boundaries

A route should translate an HTTP request into an application call and an application result into an HTTP response. It should not contain every domain rule and SQL query. A small Flask route can look like this:

Listing 8.1: A deliberately thin route boundary.

```
@app.post("/api/appointments/<int:appointment_id>/cancel")
def cancel_appointment(appointment_id: int):
    actor = current_user()
    result = appointment_service.cancel(appointment_id, actor)
    if result.is_not_found:
        return {"error": "appointment not found"}, 404
    if result.is_forbidden:
        return {"error": "not allowed"}, 403
    if result.is_conflict:
        return {"error": "appointment cannot be cancelled"}, 409
    return {"status": "cancelled"}, 200
```

The service contains the policy; the repository handles persistence; the route maps outcomes to protocol responses. Error messages should not disclose information to unauthorised callers.

8.6 Authentication and authorisation

Authentication answers who the actor is. Authorisation answers what that actor may do. A valid login does not grant access to every appointment. Enforce authorisation on the server for every protected operation. Sessions usually store a server-side identifier in a secure cookie; tokens can carry claims but require careful expiry, storage, rotation, and revocation decisions. Do not put secrets in browser code.

8.7 Validation and error handling

Validate at multiple boundaries. Client-side validation improves feedback; server-side validation is authoritative. Validate type, range, format, ownership, state, and business rules. Use status codes consistently: malformed input may be 400, unauthenticated 401, forbidden 403, missing 404, conflict 409, and unexpected failure 500. Do not expose stack traces in production.

8.8 Configuration, migrations, and deployment

Configuration varies across environments and should be supplied through environment variables or a managed configuration system. Secrets should not be committed to Git. Database schema changes need migrations that can be reviewed, applied, and rolled back or forward safely. A container packages an application and its runtime assumptions, but it does not automatically make the system secure or scalable.

A small deployment needs a process manager, TLS termination, database backups, logs, health checks, and a rollback plan. Twelve-factor principles are useful reminders about configuration, stateless processes, logs, and disposability, but they must be adapted to the system's data and operational context.

8.9 Performance and scalability

Measure the path that users experience. Network latency, database indexes, serialisation, queueing, and rendering all contribute. Horizontal scaling creates coordination questions about sessions, caches, transactions, and background jobs. Microservices are not a maturity badge; they add network boundaries, deployment complexity, and distributed failure modes. A modular monolith can be a better starting point.

8.10 The recurring case implementation

The supplied example application uses Flask and SQLite to remain small enough to inspect. It includes:

- an HTML interface and JSON endpoints;
- a domain service for appointment state;
- parameterised SQL;
- database initialisation and seed data;
- tests for domain and request behaviour;
- a clear limitation: authentication is a demonstration boundary, not production identity.

Read the directory before changing it. Trace one request from browser form to response, then add one failure test.

8.11 Project application

Show the request lifecycle in a diagram, document API contracts, and explain where authentication, authorisation, validation, persistence, and error mapping occur. When presenting the work, be prepared to answer what happens when the request is repeated, the database is unavailable, the external service times out, or the browser submits stale data.

8.12 Summary and key terms

Full-stack development connects client interaction, HTTP, server logic, domain rules, persistence, and operation. Protocol semantics and domain semantics should not be confused. Authentication and authorisation are different. Validation belongs at trust boundaries. Deployment requires configuration, migrations, observability, security, and recovery plans.

> **Key terms.** client; server; HTTP; REST; resource; JSON; frontend state; backend; domain logic; persistence; authentication; authorisation; session; token; validation; migration; container; deployment; observability; scalability.

8.13 Exercises and worked solutions

Exercise. Why is returning HTTP 200 before the database transaction commits dangerous?

Worked solution. The client receives a success claim before the durable state is known to exist. A later failure can leave the user and system with different beliefs. The response should reflect the transaction result or explicitly represent an asynchronous pending state.

Exercise. A user can view their own appointment but changes the URL to another identifier. Which security property is missing?

Worked solution. Server-side object-level authorisation is missing. Authentication identifies the caller but does not establish that the caller may access the referenced appointment. The service must check ownership or another explicit permission.

Exercise. When is a modular monolith preferable to microservices?

Worked solution. When the domain and team boundaries are still uncertain, deployment scale is modest, and a single transaction or shared model simplifies correctness. Services can be extracted later when the boundary and operational benefit justify the distributed-system cost.

8.14 Further reading

HTTP semantics are specified in RFC 9110 [**?**]; REST is an architectural style described in **?**]. Use the HTML standard [**?**] and the OWASP guidance [**?**] for implementation and security details.

Chapter 9

Databases and durable information

9.1 A motivating problem

A spreadsheet can record appointments until several people edit it at once. Then duplicates appear, a citizen's status disagrees with the staff calendar, and nobody can explain which value is authoritative. A database is not merely a faster spreadsheet. It is a system for representing, constraining, querying, and updating shared state.

> **What you will learn.** You should be able to explain the relational model; design entities, keys, and integrity constraints; use SQL selection, projection, joins, aggregation, subqueries, and views; reason about functional dependencies and normal forms; explain transactions, isolation, locks, indexes, migrations, and database security; and compare relational and non-relational models.

9.2 The five-minute explanation

A relation is a set of tuples with attributes. In a table, rows represent tuples and columns represent attributes, but a relational relation is not defined by its visual layout. A key identifies a tuple. A foreign key expresses a relationship between tables. Constraints prevent states that the domain says are invalid.

Codd's relational model separated logical data relationships from physical storage decisions [**?**]. SQL is a declarative language: the query states what result is wanted, while the database chooses an execution plan.

9.3 From domain concepts to schema

The CivicQueue schema distinguishes:

- `citizens`, identified by a surrogate identifier and containing minimal contact data;
- `services`, such as benefits advice or permit support;
- `slots`, which represent available service capacity;
- `appointments`, which connect one citizen to one slot;
- `audit_events`, which record security-relevant changes.

The schema in `examples/sql/schema.sql` can be loaded into SQLite. It uses primary keys, foreign keys, a uniqueness constraint, and a check constraint. Constraints are executable domain claims.

9.4 Relational operations and SQL

Selection filters rows; projection chooses attributes; a join combines related tuples. SQL expresses these operations:

Listing 9.1: A query that joins appointments to citizens and services.

```sql
SELECT a.id,
       c.display_name,
       s.name AS service_name,
       sl.starts_at,
       a.status
FROM appointments AS a
JOIN citizens AS c ON c.id = a.citizen_id
JOIN slots AS sl ON sl.id = a.slot_id
JOIN services AS s ON s.id = sl.service_id
WHERE a.status = 'confirmed'
ORDER BY sl.starts_at;
```

The query names the relationship conditions. An implicit join or a `SELECT *` can be shorter but may hide assumptions and create unstable interfaces. Use aliases and select the fields the client actually needs.

Aggregation uses functions such as `COUNT`, `AVG`, and `SUM`. `GROUP BY` changes the unit of analysis. If you count appointments by service, the result is no longer one row per appointment; it is one row per service. State that change explicitly.

9.5 Functional dependencies and normalisation

A functional dependency $X \to Y$ means that equal values of attributes X require equal values of attributes Y in the relation. If `service_id` determines `service_name`, storing both repeatedly in an appointment table creates update anomalies:

- update anomaly: renaming a service requires many edits;
- insertion anomaly: a service cannot be stored without an appointment;
- deletion anomaly: deleting the last appointment removes the service description.

Normalisation decomposes relations to reduce such anomalies while preserving dependencies and joinability. First, second, third normal form, and BCNF are conditions with different strengths. They are not a ritual of splitting every table. Denormalisation can be justified for measured read performance, reporting, or integration, but its duplicated facts need a consistency strategy.

9.6 Transactions and ACID

A transaction groups operations so that the database enforces a defined unit of change. ACID abbreviates atomicity, consistency, isolation, and durability. The terms do not mean that every transaction is globally correct: consistency depends on constraints and application rules; isolation has levels and trade-offs.

For rescheduling, one transaction may check the new slot, insert a new appointment, cancel or release the old slot, and write an audit event. If any required step fails, the transaction should roll back. Two concurrent transactions can still conflict unless the database and application use an appropriate constraint or locking strategy.

9.7 Isolation and concurrency

At lower isolation, transactions may see phenomena such as dirty reads, non-repeatable reads, or phantoms. At higher isolation, the database may block or abort more work. A unique constraint on an active slot can be safer than a check-then-insert sequence because the database arbitrates the race. Retry logic must distinguish transient serialization failures from permanent validation errors.

9.8 Indexes and query plans

An index is an auxiliary structure that can reduce search work at the cost of storage and write overhead. Index columns used in filters, joins, and ordering when measurement supports it. An index on a low-selectivity column may offer little benefit. Inspect query plans and benchmark representative data. Indexes do not fix an incorrect query or a missing authorisation condition.

9.9 Migrations, backups, and security

A migration changes schema in a controlled, versioned step. Backups should be encrypted, tested by restoration, and governed by retention rules. Use parameterised queries; never concatenate user input into SQL. Limit database credentials by environment and privilege. Logs can themselves contain personal data and require access control and retention decisions.

9.10 NoSQL and polyglot persistence

Document stores, key-value stores, graph databases, and wide-column systems optimise different access patterns and consistency models. A document model can be convenient for nested data, a graph model for relationship traversal, and a key-value store for predictable lookups. The choice should follow data shape, query patterns, consistency, transaction scope, operations, and team capability. “NoSQL” is not one model and does not automatically scale better.

9.11 Project application

Include an ER or relational design, functional dependencies, constraints, representative queries, transaction boundaries, indexes with evidence, migration strategy, and security analysis. When presenting the work, explain what anomaly the normalisation choice prevents and what trade-off a denormalised read model would introduce.

9.12 Summary and key terms

Databases make shared state durable and constrain its structure. The relational model supports declarative queries and integrity constraints. Normalisation is about dependencies and anomalies. Transactions coordinate changes; isolation manages concurrency trade-offs. Indexes, migrations, backups, parameterised queries, and access control are engineering parts of data management.

> **Key terms.** relation; tuple; attribute; key; foreign key; constraint; join; aggregation; functional dependency; normalisation; denormalisation; transaction; ACID; isolation; lock; index; query plan; migration; backup; NoSQL.

9.13 Exercises and worked solutions

Exercise. Why is a citizen's display name usually not a fact that should be copied into every appointment row?

Worked solution. The citizen identifier determines the current identity record, while repeated display names can become inconsistent after a correction. A report may intentionally snapshot a historical name, but then the snapshot is a different documented fact with a retention and audit rationale.

Exercise. What can go wrong with a check-then-insert reservation sequence?

Worked solution. Two concurrent transactions can both observe the slot as available and then both insert. A database uniqueness constraint, appropriate transaction isolation, or an atomic reservation operation is needed to make the invariant hold under concurrency.

Exercise. When can denormalisation be justified?

Worked solution. When a measured access pattern needs faster reads or simpler analytical queries, and the duplication has an explicit update, reconciliation, and testing strategy. It should be a response to a stated constraint, not an assumption that joins are always bad.

9.14 Further reading

For foundations, read **?**] and **?**]. PostgreSQL's documentation is a practical reference for a production relational system [**?**]; the example project uses SQLite to keep setup small.

Chapter 10

Security, testing, and operations

10.1 A motivating problem

A prototype can pass its demo while exposing appointments through predictable URLs, logging personal data, accepting forged requests, and failing silently when an email provider is unavailable. Security and operations are not separate from development; they describe what the system must preserve in an adversarial and changing environment.

> **What you will learn.** You should be able to threat-model an application; distinguish authentication, authorisation, confidentiality, integrity, and availability; recognise common web vulnerabilities; design layered tests and CI checks; and explain logging, metrics, tracing, deployment, incident response, and recovery.

10.2 The five-minute explanation

Security is the reduction of unacceptable risk to assets under threats and constraints. A threat model identifies assets, actors, trust boundaries, possible abuse paths, and mitigations. It is not a promise that no attack can succeed. Operations asks whether the system can be observed, changed, recovered, and governed after deployment.

10.3 Threat modelling CivicQueue

Assets include appointment confidentiality, slot integrity, citizen contact data, staff accounts, audit records, and service availability. Threats include account takeover, insecure direct object references, SQL injection, cross-site scripting, cross-site request forgery, malicious file input, notification abuse, and denial of service. Draw trust boundaries between browser, application, database, identity provider, and notification service.

For each risk, record:

1. asset and harm;
2. attacker capability and precondition;
3. likelihood and impact rationale;
4. preventive and detective controls;

5. residual risk and owner.

OWASP's Top 10 provides a vocabulary for common application risks, while NIST's Secure Software Development Framework connects security practices to the development lifecycle [**?** **?**].

10.4 Secure defaults

Use parameterised SQL, output encoding, server-side authorisation, secure cookie attributes, CSRF protection for state-changing browser requests, password hashing through a vetted library, TLS, rate limits, dependency updates, and least-privilege credentials. Do not invent cryptography. Do not log passwords, access tokens, or unnecessary personal data. Return generic errors to untrusted callers while preserving useful internal diagnostics.

Security controls can create usability costs. A short session timeout may reduce risk but create barriers for users with cognitive or motor impairments. The trade-off should be evaluated and supported by recovery paths.

10.5 Testing as layered evidence

A testing strategy has layers:

Unit Does one function or domain object satisfy its contract?

Integration Do components interact correctly with a real or controlled database and external boundary?

System Does the deployed application perform a complete user task?

Property and invariant Does a rule hold across generated inputs?

Security Can an unauthorised actor violate confidentiality or integrity?

Usability and accessibility Can intended users complete tasks and recover from errors?

A high percentage of unit coverage does not imply high system quality. Tests should be selected from risks and requirements, not from a desire to maximise a single number.

10.6 Continuous integration and delivery

A CI pipeline should fail clearly and reproducibly. Typical steps are dependency installation from a lock or constrained file, linting, type checks, unit tests, integration tests, security checks, packaging, and build verification. A deployment pipeline promotes an immutable artefact through environments with approvals appropriate to risk. Database migrations and rollback plans belong in the release design.

10.7 Observability and reliability

Logs record events; metrics aggregate measurements; traces follow a request across components. Use correlation identifiers rather than personal identifiers where possible. Define

service-level indicators such as successful booking rate and latency. An alert should have an owner and an action, not merely report that a graph changed.

Reliability includes failure handling: timeouts, retries with backoff, idempotency keys, circuit breakers, graceful degradation, backup restoration, and incident review. Retrying a non-idempotent reservation can create duplicates. Make the operation's semantics explicit.

10.8 Performance and capacity

Capacity planning estimates demand and resource limits. A queue, database connection pool, and notification provider can each become the bottleneck. Load tests should use representative traffic and safe data. Report uncertainty and environmental limits. Do not interpret a local benchmark as a guarantee about production.

10.9 Accessibility and privacy in operation

Accessibility is not finished at the interface layer if an outage removes the only accessible route. Privacy includes logs, backups, analytics, monitoring, support access, and data deletion. Data retention should be justified by purpose and legal context. A system that is secure from attackers but unusable by the intended population has failed a central requirement.

10.10 Project application

Include a threat model, test strategy, CI evidence, deployment diagram, operational assumptions, and recovery scenario. Demonstrate one failure intentionally: disconnect the database, submit an invalid transition, or replay a request. When documenting the work, distinguish a vulnerability you mitigated from a risk you merely documented.

10.11 Summary and key terms

Security and operations extend engineering beyond the happy path. Threat modelling connects assets to risks and controls. Authentication is not authorisation. Layered tests provide different evidence. Observability, reliability, deployment, backup, and recovery are design concerns. Privacy and accessibility continue through logs and operations.

> **Key terms.** threat model; asset; trust boundary; authentication; authorisation; confidentiality; integrity; availability; SQL injection; XSS; CSRF; least privilege; unit test; integration test; CI/CD; observability; log; metric; trace; idempotency; incident response.

10.12 Exercises and worked solutions

Exercise. A request to cancel an appointment is retried after a timeout. What should the design consider?

Worked solution. The server may have completed the first request even though the client did not receive the response. The operation should be idempotent or use an idempotency key so that a retry does not create a second effect. The response should represent the current state consistently.

Exercise. Why is “we use HTTPS” not a complete security argument?

Worked solution. HTTPS protects transport under its assumptions, but it does not prevent broken authorisation, injection, insecure cookies, weak passwords, malicious application logic, leaked secrets, or harmful logging. Security is a set of controls across the system.

Exercise. Give one example of an operational metric that is not merely server CPU.

Worked solution. The proportion of successful booking attempts, the p95 confirmation latency, notification delivery failure rate, or the number of unauthorised access attempts may be more directly connected to user and security outcomes than CPU alone.

10.13 Further reading

Use OWASP’s Top 10 and Application Security Verification Standard as practical starting points [**?**]; use NIST SP 800-218 for secure development practices [**?**]. Reliability concepts should be connected to the system’s actual failure modes rather than copied as a checklist.

Part III

People, organisations, and development practice

Chapter 11

Interaction design and human–computer interaction

11.1 A motivating problem

A CivicQueue prototype contains every required field and still confuses users. Citizens do not know whether a slot is reserved, staff cannot tell which changes need approval, and an error message says only “invalid input”. The problem is not visual taste. It is the relationship between a person, a goal, a representation, an action, and feedback.

> **What you will learn.** You should be able to explain perception, attention, memory, mental models, affordances, signifiers, feedback, constraints, mapping, cognitive load, errors and recovery, user-centred design, participatory methods, information architecture, interaction flows, and accessibility; and distinguish a design preference from an empirical claim.

11.2 The five-minute explanation

Interaction design shapes what people can do, what they think they can do, and what the system communicates about what happened. A good interface does not eliminate human judgment; it supports people in forming an accurate working understanding and recovering when things go wrong.

> **In plain language.** If a button looks available, users may expect it to work. If a change is saved but no feedback appears, users may repeat the action. If the only error message is red text, a person who cannot see colour may miss it. Design is the management of expectations and consequences.

11.3 Human cognition and action

People have limited attention and working memory. They use recognition, prior knowledge, external cues, and routines. A mental model is the user’s working explanation of how a system behaves. Interfaces can support or disrupt that model. The goal is not to memorise a fixed number of interface rules; it is to reduce unnecessary cognitive work while making important distinctions visible.

Affordance describes what an object allows; a signifier communicates how it may be used. A flat rectangle may afford clicking in software but needs a signifier such as label, focus style, and pointer behaviour. Feedback tells the user what happened. Constraints prevent invalid actions or make them harder. Mapping connects controls to effects. These concepts are related but not interchangeable.

11.4 User-centred and participatory design

User-centred design is an iterative process involving users, tasks, context, prototyping, and evaluation. Participatory design gives affected people a role in shaping the problem and solution, not only in testing a finished interface. Neither approach guarantees fairness: the recruited users may not represent people with less power, and organisational constraints may limit which suggestions can be adopted.

Useful methods include:

Contextual inquiry Observe work in context while asking about actions and exceptions.

Interview Elicit experiences, goals, language, and interpretations; do not treat reported preference as direct evidence of behaviour.

Observation Record what happens, including workarounds, interruptions, and artefacts.

Task analysis Decompose a goal into actions, decisions, information needs, and failure recovery.

Persona A synthesis of research patterns, not a fictional stereotype.

Scenario A situated narrative that makes goals, context, and constraints concrete.

11.5 Information architecture and interaction flows

Information architecture organises content and actions so that users can find, interpret, and navigate them. For CivicQueue, “My appointments”, “Available services”, and “Help and alternatives” may be more understandable than internal database categories. An interaction flow should include the start state, action, system feedback, alternative paths, error recovery, and end state.

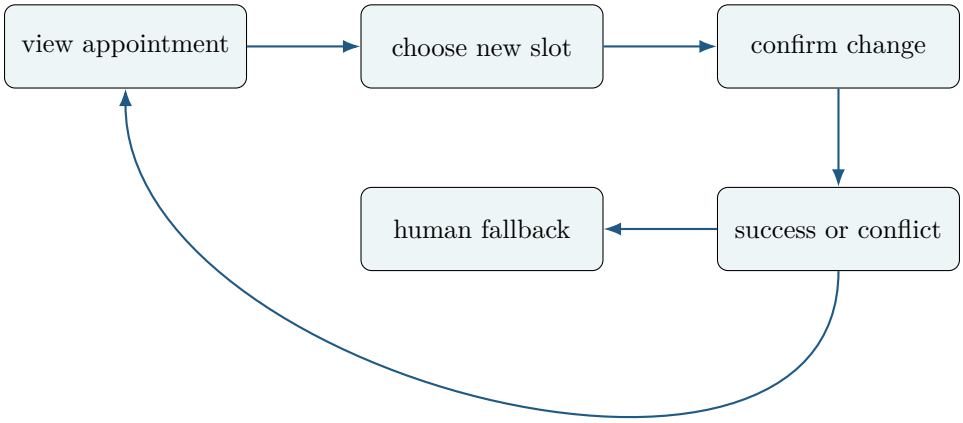

Figure 11.1: An interaction flow includes success, conflict, and recovery paths.

11.6 Prototyping

A paper sketch, wireframe, clickable prototype, and working system answer different questions. Low-fidelity prototypes make information architecture and task flow cheap to change. High-fidelity prototypes can test visual hierarchy and interaction timing but may cause stakeholders to infer that backend behaviour already exists. Label what is real, simulated, or intentionally omitted.

11.7 Visual hierarchy and inclusive design

Typography, spacing, grouping, contrast, and alignment communicate structure. Aesthetic consistency can support recognition, but preference is not usability evidence. Accessibility includes semantic structure, keyboard operation, focus visibility, contrast, text resizing, captions, error identification, and compatibility with assistive technology. WCAG 2.2 provides testable success criteria, but conformance alone does not prove that a particular service is usable in context [**?**].

Inclusive design asks who is absent from the default. Consider language, literacy, motor and sensory variation, cognitive load, device access, connectivity, age, and institutional trust. A digital-only process may be efficient for one group and exclusionary for another.

11.8 Ethics, privacy, and dark patterns

An interface can manipulate through hidden costs, confusing consent, forced continuity, or defaults that benefit the provider over the user. Dark-pattern analysis asks who gains from the friction and who bears it. Privacy-friendly design uses data minimisation, clear purposes, granular choices, and deletion or correction routes. A consent checkbox cannot repair an interface that makes refusal practically impossible.

11.9 Project application

Ground design decisions in observed tasks and stakeholder language. Include a rationale for the interaction structure, a prototype history, accessibility considerations, and a map from problem to design response. When presenting the work, distinguish what users said, what they did, what you inferred, and what remains unknown.

11.10 Summary and key terms

Interaction design manages goals, actions, representations, feedback, and recovery. Cognitive concepts guide hypotheses rather than replace evaluation. User-centred and participatory methods reveal context but require careful sampling and interpretation. Prototypes have different purposes. Accessibility and privacy are design conditions, not decorative additions.

> **Key terms.** interaction design; HCI; mental model; affordance; signifier; feedback; constraint; mapping; cognitive load; user-centred design; participatory design; contextual inquiry; persona; scenario; information architecture; prototype; inclusive design; dark pattern.

11.11 Exercises and worked solutions

Exercise. A user submits a rescheduling form and the page simply returns to the top. Which design concepts are missing?

Worked solution. The system lacks visible and programmatically available feedback, error identification, focus management, and a clear mapping between action and outcome. The solution may require both interface changes and a backend response that distinguishes success from validation failure.

Exercise. Why is a persona not evidence that every user has the persona's preferences?

Worked solution. A persona is a synthesis of patterns in collected research. It can focus design attention, but it abstracts away variation and depends on sampling and interpretation. It should be accompanied by research notes and limitations.

Exercise. Give one reason a paper prototype can be more informative than a polished prototype.

Worked solution. It invites stakeholders to criticise structure and content without assuming that implementation is fixed. It is cheaper to change and less likely to hide uncertainty behind visual polish.

11.12 Further reading

Read **?**] for concepts of action and feedback, and **?**] for interaction-design methods. Adapt the methods to the question, users, risks, and evidence available in the setting you are studying.

Chapter 12

Evaluating interfaces and interactive systems

12.1 A motivating problem

The CivicQueue team says, “users liked the new design”. That sentence may mean that participants praised the visual style, completed tasks faster, felt less anxious, or merely wanted to be polite. Evaluation turns a design claim into a question with a method, measure, sample, and interpretation.

> **What you will learn.** You should be able to choose among heuristic evaluation, cognitive walkthroughs, interviews, observation, think-aloud studies, questionnaires, and experiments; define usability and accessibility measures; analyse qualitative and quantitative evidence; discuss reliability and validity; and report limitations without inflating conclusions.

12.2 The five-minute explanation

Evaluation asks whether an artefact or design performs in relation to a purpose and criterion. Usability commonly includes effectiveness, efficiency, and satisfaction in a specified context. User experience is broader and may include emotion, meaning, trust, and long-term consequences. Accessibility concerns whether people with diverse abilities can perceive, operate, understand, and robustly use the system. Aesthetic preference is a different construct.

A claim such as “the new design is better” is incomplete. Better for whom, at which task, compared with what, measured how, and with what uncertainty?

12.3 Inspection methods

Heuristic evaluation asks trained evaluators to inspect an interface against principles such as visibility of system status, match with the real world, user control, consistency, error prevention, recognition rather than recall, flexibility, minimalist design, error recovery, and help. Heuristics help find problems; they do not measure actual user performance [**?**].

A cognitive walkthrough examines whether a new or infrequent user can form the correct goal, notice the action, associate it with the goal, and understand the feedback. It is useful for task learnability, but its result depends on the evaluator's assumptions.

12.4 User studies

A think-aloud study asks participants to verbalise their reasoning while performing tasks. It can expose mental models and uncertainty, but talking may change performance. Observation reveals behaviour and workarounds. Interviews reveal accounts and interpretations. Use a task script, consent procedure, recording plan, and stopping rule appropriate to the risk.

A small formative study can identify severe problems without estimating a population parameter. If the purpose is to compare two interfaces, pre-specify tasks, outcomes, analysis, and exclusions. A/B testing can provide useful evidence only when exposure, randomisation, interference, ethics, and practical significance are considered.

12.5 Measures

Task success may be binary or defined by a rubric. Time-on-task is sensitive to task definition and interruptions. Error rate requires a consistent error coding scheme. Satisfaction scales require instructions, scoring, and interpretation. The System Usability Scale has ten items and a specific scoring procedure; its score is a measure under that instrument, not a universal truth [**?**].

If n participants have success indicators $Y_i \in \{0, 1\}$, the observed success proportion is

$$\hat{p} = \frac{1}{n} \sum_{i=1}^{n} Y_i.$$

The symbols mean: n is the number of observed participants, Y_i is participant i's coded outcome, and $\hat{p}$ is the sample proportion. It estimates a target only under assumptions about sampling, task definition, and measurement. Report the denominator and uncertainty; do not present a percentage without context.

12.6 Qualitative coding

Qualitative analysis is not a pile of quotations. Define the unit of analysis, code relevant material, compare cases, revise the codebook, and explain how themes were constructed. A theme is a patterned meaning relevant to the research question. Braun and Clarke's thematic-analysis framework is influential, but different epistemological positions imply different standards for reflexivity and claims [**?**].

12.7 Reliability and validity

Reliability concerns consistency of a measurement or procedure. Validity concerns whether the measure supports the intended interpretation. A reliable measure can be invalid if it consistently measures the wrong construct. Internal validity concerns alternative

explanations within a study. External validity concerns transfer or generalisation. Construct validity concerns the relation between operationalisation and concept.

Evaluation validity is therefore an argument. Document the construct, measure, participants, procedure, analysis, and limitations.

12.8 Accessibility evaluation

Combine automated checks, manual keyboard and screen-reader inspection, and testing with people who use relevant assistive technologies. Automated tools can detect some structural problems and miss context, task clarity, focus order, or confusing language. Test the complete task, including errors, timeouts, and human fallback.

12.9 Project application

Before recruiting, write an evaluation protocol. State the research question, design, population, sampling, task, measures, analysis, ethical safeguards, and decision rule. When documenting the work, distinguish formative findings from comparative evidence. When presenting it, explain one plausible alternative interpretation of your result.

12.10 Summary and key terms

Evaluation makes design claims answerable to evidence. Inspection methods and user studies answer different questions. Measures require operational definitions and denominators. Qualitative coding requires an analytic procedure. Reliability, construct validity, internal validity, external validity, and accessibility are distinct arguments. Evidence supports bounded interpretations, not automatic claims of superiority.

> **Key terms.** usability; user experience; accessibility; heuristic evaluation; cognitive walkthrough; think-aloud; task success; error rate; System Usability Scale; qualitative coding; theme; reliability; validity; construct; internal validity; external validity; formative evaluation.

12.11 Exercises and worked solutions

Exercise. Why would a heuristic evaluation and a usability test find different problems?

Worked solution. Heuristic evaluation uses trained inspection against principles and can identify consistency, feedback, or error-prevention problems without recruiting users. A usability test observes people performing tasks and can reveal misunderstandings, workarounds, and timing that the evaluator did not anticipate. Each method has different blind spots.

Exercise. A study reports that 9 of 10 participants succeeded. What can you conclude?

Worked solution. You can report that 9 of the 10 observed participants succeeded under the specified procedure. You cannot automatically conclude that 90 percent of the target population would succeed, that the interface caused success, or that it is better than an alternative.

Exercise. Give one example of a reliable but invalid measure in an interface study.

Worked solution. A timer that consistently records the time from page load rather than task start may be reliable but invalid for task duration. A questionnaire that consistently captures visual appeal when the claim concerns trust is another possibility.

12.12 Further reading

For HCI research methods, see **?**]; for empirical software studies, see **?**]. Use WCAG as a technical reference, while recognising that conformance is not the same as contextual usability.

Chapter 13

Information and organisation

13.1 A motivating problem

A municipality buys a case-management platform because it promises efficiency. After deployment, employees keep a private spreadsheet, citizens repeat information at multiple desks, and managers cannot agree on what a "resolved case" means. The implementation did not fail because people irrationally resisted technology. It failed because the system changed work, authority, information, and accountability.

> **What you will learn.** You should be able to analyse information systems as sociotechnical systems; distinguish data, information, knowledge, and organisational routines; explain business processes and system types; analyse value, adoption, power, governance, privacy, procurement, migration, legacy, and implementation; and evaluate organisational benefits without treating technology as an isolated optimisation.

13.2 The five-minute explanation

An information system is a coordinated arrangement of people, practices, data, technologies, and rules that produces and uses information. Its value is not located solely in software features. A system can create value by supporting coordination, decisions, accountability, service access, learning, or control. It can also redistribute work and power.

Data is a recorded distinction. Information is data interpreted in a context. Knowledge includes justified understanding and the ability to act. These categories are useful teaching distinctions, not a universal ladder.

13.3 Processes and system types

A business process is a connected set of activities that transforms inputs into outcomes for a stakeholder. Model both the official process and the workarounds. Transaction-processing systems record routine events. Management-information systems aggregate operations for monitoring. Decision-support systems help analyse alternatives. ERP integrates organisational functions; CRM coordinates relationships with customers or citizens;

supply-chain systems coordinate material, information, and financial flows. Platform systems mediate interactions among groups and often create governance questions.

The same software can serve several roles depending on how it is embedded. A dashboard that supports local learning differs from one used for punitive surveillance even if the charts are identical.

13.4 Value and success

A value claim should identify beneficiary, mechanism, outcome, comparison, and cost. The DeLone–McLean model distinguishes dimensions such as system quality, information quality, use, user satisfaction, and net benefits, while recognising that success is multidimensional [**?**]. Technology acceptance models can explain some adoption beliefs, but acceptance is not the same as beneficial use [**?** **?**].

For CivicQueue, possible benefits include fewer unnecessary telephone calls, easier rescheduling, better slot utilisation, and more predictable staff work. Possible costs include exclusion, training time, notification burden, privacy risk, and duplicated administration. Evaluate both.

13.5 Implementation, adoption, and resistance

Implementation is a change process. It includes procurement, configuration, integration, migration, training, support, governance, incentives, and learning. Resistance may signal workload, loss of discretion, mistrust, poor fit, or a legitimate concern. Shadow IT can be a workaround, an innovation site, a security risk, or all three.

Sociotechnical perspectives reject the idea that a technology has one fixed effect independent of use. Orlikowski's work on the duality of technology shows how structures are both enabled and reproduced through practice [**?**]. Power and politics influence which requirements are accepted and whose failure becomes visible [**?**].

13.6 Governance, privacy, and data

Data governance defines ownership, quality responsibilities, access, retention, provenance, and decision rights. GDPR principles such as purpose limitation, data minimisation, accuracy, storage limitation, integrity, and accountability should be connected to actual flows and roles, not cited as abstract slogans [**?**].

A data map should identify collection, processing, sharing, storage, retention, deletion, and user rights. In CivicQueue, an audit event may improve accountability but also create sensitive records. A notification provider may receive contact data. A data-protection impact assessment may be appropriate when processing creates significant risks.

13.7 Procurement, integration, and legacy

Vendor selection should evaluate functional fit, interoperability, exit options, security, accessibility, service levels, total cost, data portability, and organisational capability. Integration can fail through incompatible identifiers, timing assumptions, semantic mismatch, or ownership ambiguity. A legacy system is not merely old code; it may contain institutional knowledge and operational dependencies. Migration should include data profiling, mapping, validation, rehearsal, rollback, and communication.

13.8 Organisational benefit evaluation

A before-and-after change can be descriptive but is vulnerable to time trends, selection, and concurrent changes. A process measure such as average handling time is not necessarily a service benefit. Combine quantitative indicators with observation and interviews, and specify whose perspective defines benefit.

13.9 Project application

Draw the technical architecture and the organisational process side by side. Identify which roles gain or lose discretion, which routines change, what training is needed, and which benefits are plausible but untested. When presenting the work, explain why a technically elegant solution might be rejected by the organisation and what alternative implementation path you would propose.

13.10 Summary and key terms

Information systems are sociotechnical arrangements. Data gains meaning through context and practice. System value is multidimensional and contested. Adoption, resistance, shadow IT, power, governance, migration, and legacy are part of system development. Evaluation must include costs, distributional effects, and alternative explanations.

> **Key terms.** information system; data; information; knowledge; business process; transaction processing; MIS; decision support; ERP; CRM; platform; value creation; adoption; resistance; shadow IT; governance; GDPR; migration; legacy; sociotechnical system.

13.11 Exercises and worked solutions

Exercise. Why is a private spreadsheet not automatically evidence that the official system is unnecessary?

Worked solution. It may be a workaround for a missing view, a temporary coordination tool, or an unsafe duplicate source. It reveals a mismatch between system and work, but its

presence alone does not establish which system should be authoritative or what risks the spreadsheet creates.

Exercise. Give one example of a technical quality and one organisational quality for CivicQueue.

Worked solution. Technical quality might be transaction integrity so that a slot cannot be double-booked. Organisational quality might be that staff can resolve exceptions without duplicating data entry. The two interact but are not the same measure.

Exercise. How can implementation change power?

Worked solution. A system can make some decisions visible to managers, constrain frontline discretion, allocate work automatically, or give citizens new ways to challenge records. These changes can redistribute authority even when the software is described as administrative.

13.12 Further reading

For further reading on information-system types, organisational value, implementation problems, and research methods, compare **?**] with **?**]. Together they show why organisational problems cannot be reduced to technical requirements.

Chapter 14

Agile development, PBL, and collaborative practice

14.1 A motivating problem

A group has a promising idea, four different assumptions, and six weeks before a project demonstration. One student writes most of the code, another produces diagrams at the end, and a third discovers that the target users were unavailable. The group has been busy but has not created a shared learning process.

> **What you will learn.** You should be able to explain iterative and incremental development; compare Scrum, Kanban, XP, plan-driven, and hybrid approaches; use backlogs, reviews, retrospectives, definitions of done, and continuous integration critically; explain Conway's and Brooks's laws; and apply problem- and project-based learning principles to team work and reflection.

14.2 The five-minute explanation

Agile development is a family of principles and practices for learning and delivering under changing conditions. It is not a synonym for no planning, no documentation, or permanent urgency. Iteration means revisiting decisions; increment means producing a usable addition. A team can iterate without producing a coherent increment, or produce increments without learning from users.

Problem-based learning emphasises problem-oriented project work, student responsibility, collaboration, and reflection [**?**]. That overlaps with agile practices but is not identical to Scrum. PBL is an educational model; Scrum is a work-management framework.

14.3 Scrum, Kanban, and XP

Scrum organises work around a product goal, backlog, sprint, review, and retrospective. Kanban visualises flow and limits work in progress. Extreme Programming emphasises technical practices such as test-driven development, pair programming, simple design, continuous integration, and collective ownership. Use the practice that addresses a real coordination or quality problem.

A backlog is a changing model of possible work. A story should describe a user or system goal, but a story is not a full requirement specification. Acceptance criteria, constraints, data assumptions, and risks still need documentation. The Definition of Done should include the quality conditions that make an increment usable, tested, reviewable, and explainable.

14.4 Planning and estimation

Estimation is a forecast under uncertainty, not a moral judgement about effort. Story points can support relative discussion but do not become units of time by magic. A plan should identify dependencies, unknowns, decision deadlines, and a smallest useful outcome. Burn-down charts are descriptive; they do not diagnose whether the right work is being done.

14.5 Technical practices

Continuous integration reduces the cost of discovering incompatible changes. Pairing and review distribute knowledge. Automated tests make refactoring safer. Documentation should capture domain terms, decisions, interfaces, setup, risk, and operational assumptions. In regulated or safety-critical settings, agile practices may be combined with formal requirements, traceability, verification, and change control.

14.6 Conway's and Brooks's laws

Conway's law is an empirical observation that system designs tend to reflect communication structures. It is a useful hypothesis about organisational architecture, not a deterministic law. Brooks's law states that adding people to a late software project can make it later because communication and onboarding costs increase. Both laws point to coordination costs that a task board can hide.

Psychological safety affects whether people report uncertainty, challenge a design, admit mistakes, and ask for help. A retrospective should examine work conditions and system outcomes, not turn into blame.

14.7 PBL project practice

A disciplined project-based cycle can include:

1. formulate and delimit a problem;
2. identify learning needs and relevant literature;
3. divide responsibility while preserving shared understanding;
4. develop artefacts and evidence iteratively;
5. use supervision to test interpretations;
6. analyse the product and the process;

7. communicate results in a written account, demonstration, and clear presentation.

Group work is not simply parallel individual work. Interfaces among members, integration, decisions, and responsibility for the whole argument must be managed.

14.8 Reflection and division of responsibility

A process reflection should use evidence: meeting notes, decision logs, Git history, issue flow, peer feedback, and changes in scope. Avoid confessional writing without analysis. Explain what happened, why it happened, what effect it had, and what practice would change.

One person may lead database work, another interaction research, and another deployment, but everyone should understand the system boundary, research question, major decisions, and limitations. Specialisation without integration creates fragile work.

14.9 Presenting and reviewing the work

A convincing project makes its problem, decisions, evidence, limitations, and contribution visible. Each contributor should be able to explain both their own work and the whole project's argument. Demonstrations should be rehearsed with failure paths, not only the ideal click sequence.

14.10 Summary and key terms

Agile methods manage learning and delivery under uncertainty but are not universally superior. Scrum, Kanban, XP, and plan-driven methods solve different coordination problems. PBL is an educational model built around problem-oriented project work and reflection. Estimation, documentation, review, CI, psychological safety, and integration determine whether a group can create shared evidence.

> **Key terms.** agile; iterative; incremental; Scrum; Kanban; XP; backlog; sprint; review; retrospective; Definition of Done; estimation; continuous integration; Conway's law; Brooks's law; psychological safety; PBL; supervision; reflection.

14.11 Exercises and worked solutions

Exercise. Why can a team follow Scrum and still fail to learn?

Worked solution. It can complete planned tickets without testing assumptions, involving users, measuring outcomes, or revising the problem formulation. Ceremonies organise communication; they do not guarantee inquiry or useful increments.

Exercise. What should a Definition of Done contain for a CivicQueue feature?

Worked solution. It might require implemented domain logic, tests for success and conflict, reviewed code, updated API and data documentation, accessible feedback, security checks, and a demonstrated path in a test environment. The exact definition depends on risk and project scope.

Exercise. How can division of responsibility improve and harm a project?

Worked solution. It can reduce overload and allow focused expertise. It can harm the project when knowledge becomes isolated, integration is postponed, or one person's work cannot be explained or tested by the group. Rotate review and preserve a shared system model.

14.12 Further reading

Read the Agile Manifesto [**?**], the Scrum Guide [**?**], and the research on alignment between PBL and assessment [**?**]. Do not treat a framework's vocabulary as evidence that its use caused project success.

Part IV

Evidence, AI, and independent work

Chapter 15

Research on interactive systems

15.1 A motivating problem

The CivicQueue team builds a working prototype and calls it a research contribution. A supervisor asks: What is the research question? Which concept is being studied? What evidence would support the claim? What alternative explanation remains? A functioning artefact can be valuable without producing generalisable knowledge; a research contribution requires a defensible relation among question, theory, method, evidence, and claim.

> **What you will learn.** You should be able to formulate researchable questions; search and synthesise literature; distinguish qualitative, quantitative, mixed-methods, case, experimental, and design-science approaches; reason about sampling, measurement, reliability, validity, ethics, and reproducibility; and report bounded claims.

15.2 The five-minute explanation

Research is a systematic, transparent inquiry aimed at answering a question or contributing knowledge. The word systematic does not mean mechanical. It means that the route from question to conclusion is sufficiently explicit for others to evaluate. A project can build, describe, explain, predict, interpret, or evaluate; these purposes imply different designs.

A product asks “can we build this?” Research asks “what can we justifiably claim after building or studying it?” The two may be integrated, but they should not be conflated.

15.3 Questions, concepts, and frameworks

A research question identifies a phenomenon, population or context, relation, process, or comparison. Concepts such as usability, trust, adoption, privacy, workload, and organisational value need definitions. A conceptual framework states how the selected concepts are related and which mechanisms or assumptions guide inquiry.

A literature review is an argument about what is known, how it is known, where findings disagree, and what remains open. Search terms, databases, inclusion criteria, and citation chaining should be recorded. A list of summaries is not a synthesis.

15.4 Qualitative, quantitative, and mixed methods

Qualitative methods study meaning, practice, interpretation, and context using interviews, observation, documents, interaction traces, and coding. Quantitative methods represent variables numerically and estimate patterns, differences, associations, or uncertainty. Mixed methods combines approaches when their integration answers a question better than either alone.

A method is not defined by the type of data alone. A log can support quantitative counts or qualitative interpretation. An interview can be analysed descriptively or used to develop a measure. The research question and epistemological assumptions matter.

15.5 Study designs

Case study An in-depth investigation of a contemporary phenomenon in context, with explicit case and evidence boundaries [**?**].

Experiment Manipulates an intervention and compares outcomes under specified assignment and measurement conditions.

Survey Measures reported characteristics or attitudes in a sample; validity depends on question wording, sampling, non-response, and construct measurement.

Usability study Observes or evaluates interaction in relation to tasks and users.

Design science Builds and evaluates an artefact while contributing design knowledge, constructs, methods, or theory.

Systematic synthesis Uses a transparent search and selection process to combine prior evidence.

A design can be rigorous without being large. Rigour comes from fit, transparency, and justified inference.

15.6 Sampling and measurement

A population is the set to which a claim refers. A sample is the observed subset. Probability sampling supports some forms of generalisation; purposive sampling can support depth and conceptual relevance. Convenience sampling is not automatically invalid, but its limitations must shape the claim.

Measurement maps a construct to an observation. If *trust* is measured by one agreement item, the item is not identical to the construct. Reliability, content validity, construct validity, and criterion validity are different considerations. A precise number can be a poor measure.

15.7 Uncertainty and effect estimates

For an observed mean $\bar{x}$ from n observations, the standard error describes sampling variability under a model. A simple confidence interval may be written

$$\bar{x} \pm t^* \frac{s}{\sqrt{n}},$$

where $\bar{x}$ is the sample mean, s the sample standard deviation, n the sample size, and t^* a critical value determined by the chosen interval and degrees of freedom. This interval is not a probability statement that the fixed parameter lies inside after the data are observed. Its interpretation depends on the sampling and model assumptions.

For a comparative study, report the difference, uncertainty, data-generating context, and practical meaning. A small p-value does not establish a large or important effect; a non-significant result does not prove no effect.

15.8 Validity and causal claims

Internal validity concerns whether the study supports the claimed explanation rather than a confounder, selection effect, history, measurement change, or attrition. External validity concerns transfer. Construct validity concerns operationalisation. Statistical conclusion validity concerns whether the analysis supports the stated pattern.

Causal language requires a counterfactual comparison: what would have happened to the same units under another condition. A before-and-after change in CivicQueue may reflect seasonal demand, staffing, policy, or regression to the mean. Use a design appropriate to the claim, and prefer descriptive language when causal identification is weak.

15.9 Ethics, consent, and reproducibility

Research involving people requires appropriate consent, data minimisation, confidentiality, risk assessment, and a plan for withdrawal where applicable. Ethical review is not a formatting step. Participants may be employees, citizens, or learners with unequal power.

Reproducibility benefits from versioned code, data dictionaries, analysis scripts, preregistered decisions when appropriate, documented exclusions, and an account of missing data. Privacy may require synthetic data, controlled access, or non-release of raw records. Openness is constrained by rights and risk.

15.10 Academic argumentation

A research argument has a claim, reasons, evidence, assumptions, and limitations. Separate findings from interpretation. Do not let a limitation paragraph erase an overbroad headline. Use tables that connect research questions to data and analysis.

15.11 Project application

A project proposal should state the problem, question, concepts, literature, design, artefact if any, data, analysis, ethics, timeline, and intended contribution. When presenting it, explain why a different method was not chosen and what evidence would falsify or weaken the conclusion.

15.12 Summary and key terms

Research produces defensible claims, not merely software. Questions, concepts, frameworks, methods, measures, samples, and analyses must fit. Validity concerns the inference, not only the instrument. Causal claims require counterfactual reasoning. Ethics, uncertainty, reproducibility, and limitations are part of the result.

> **Key terms.** research question; conceptual framework; literature review; qualitative; quantitative; mixed methods; case study; experiment; survey; design science; population; sample; measurement; reliability; validity; uncertainty; causal claim; reproducibility; reflexivity.

15.13 Exercises and worked solutions

Exercise. Turn “does CivicQueue work?” into a descriptive and a comparative research question.

Worked solution. Descriptive: “How do citizens and staff use the rescheduling workflow during the first four weeks of a pilot?” Comparative: “Compared with the existing telephone process, does access to CivicQueue change rescheduling completion time or error rate under specified conditions?” The comparative question still needs a design and measures.

Exercise. Why can a case study be rigorous without a representative sample?

Worked solution. Its purpose may be analytical depth and explanation of a bounded case rather than population estimation. Rigour requires a clear case boundary, evidence chain, alternative explanations, and justified transfer claims.

Exercise. What is the difference between an artefact and a contribution?

Worked solution. The artefact is what was built. The contribution is what others can learn from the work, such as a validated design principle, an empirical finding, a method, a theoretical refinement, or a reusable system component. The relation must be argued and evidenced.

15.14 Further reading

For HCI methods, see **?**]; for software-engineering experimentation, see **?**]; for causal and quasi-experimental design, see **?**].

Chapter 16

Human-centred artificial intelligence

16.1 A motivating problem

A municipality considers an AI assistant that predicts which appointment reminders need personal follow-up. The model is accurate on a validation set, but staff do not know when to trust it, citizens cannot challenge a classification, and the training data reflects unequal historical contact. The central design question is not whether the model is intelligent. It is whether the complete human–AI system supports a legitimate purpose with calibrated uncertainty, human control, and accountable failure recovery.

> **What you will learn.** You should be able to explain supervised and unsupervised learning, training/validation/test separation, neural networks, language models, probabilistic outputs, uncertainty, explainability, bias, fairness, privacy, human-in-the-loop interaction, evaluation, and relevant European regulation; and distinguish model performance from system-level benefit.

16.2 The five-minute explanation

Machine learning estimates patterns from data rather than receiving every rule explicitly. In supervised learning, examples have target labels. In unsupervised learning, the system searches for structure without a supplied target. A model can be useful for prediction without representing a causal mechanism or possessing human understanding.

The training set is used to fit parameters. Validation data supports model or design choices. A held-out test set estimates performance after those choices. Reusing test data for tuning makes the estimate optimistic. A deployment setting may differ from all three.

16.3 A small mathematical model

A classifier maps an input vector x to a predicted label or probability. Logistic regression can represent

$$p(y = 1 \mid x) = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

where x is a vector of features, w a vector of learned weights, b an intercept, and σ maps a real score to a number between 0 and 1. The value is a model output under its training and calibration assumptions, not an intrinsic probability that the individual truly belongs to a category.

A neural network composes parameterised functions. Training typically minimises a loss over examples using an optimisation algorithm. The resulting parameters can fit patterns, exploit shortcuts, or encode artefacts of the data.

16.4 Neural networks and language models

A large language model predicts tokens in context. Transformer architectures use attention mechanisms to combine information across a sequence [**?**]. Scale and training data can produce broad capabilities, but fluent output is not proof of truth, grounded reasoning, or stable intention. The system generates likely continuations conditioned on input and internal parameters.

Generative models can hallucinate, reproduce sensitive data, amplify stereotypes, and fail under distribution shift. Their usefulness depends on the surrounding workflow: grounding, verification, access controls, user interface, monitoring, and a human decision process.

16.5 Uncertainty and calibration

If a model outputs 0.8 for many cases, calibration asks whether about 80 percent of such cases have the target outcome under the evaluation distribution. Accuracy and calibration are different. Uncertainty can arise from noisy data, limited coverage, ambiguity, distribution shift, or model instability. A confidence score is not automatically calibrated uncertainty.

A human-centred interface should show what the output means, what data it used where appropriate, what it does not know, how to correct it, and what happens when it fails. Explanations can support debugging or user control without providing a true mechanistic account of the model.

16.6 Human-in-the-loop systems

A human-in-the-loop may approve, correct, contextualise, contest, or override a model. Adding a human does not automatically make a system safe. Automation bias can lead people to accept recommendations without scrutiny; alert fatigue can make them ignore everything. Design the task, authority, workload, evidence display, escalation path, and audit trail together.

Evaluate the combined system. Relevant measures include model discrimination and calibration, human decision quality, time, workload, disagreement patterns, appeal outcomes, subgroup performance, and consequences of error.

16.7 Bias and fairness

Bias can enter through historical labels, sampling, measurement, missingness, proxy variables, deployment, or feedback loops. Fairness criteria can conflict when base rates differ and decisions are imperfect. A single fairness metric cannot settle a normative question. State the protected groups, decision context, harms, threshold, and trade-offs.

Privacy includes training data, prompts, outputs, logs, model updates, and third-party processing. Data minimisation and purpose limitation remain relevant. A model card or data documentation can make intended use, limitations, evaluation data, and risks inspectable [**?** **?**].

16.8 European regulation and responsibility

The EU Artificial Intelligence Act establishes harmonised rules using a risk-based framework; its application depends on system role, provider/deployer obligations, and the law's implementation timeline [**?**]. The regulation does not replace technical risk analysis, professional judgment, or rights-based evaluation. The GDPR may also apply to personal data [**?**].

16.9 Project application

Define the human task before choosing a model. Specify the data-generating process, split strategy, baseline, metrics, subgroup analysis, uncertainty, interface, human override, logging, and failure recovery. When presenting the work, explain what the model cannot establish and why a simpler rule, search system, or non-AI workflow might be preferable.

16.10 Summary and key terms

AI systems estimate patterns under data and modelling assumptions. Training, validation, and test data must remain conceptually separate. Probabilistic outputs require calibration and interpretation. Human-in-the-loop design can fail through automation bias or weak authority. Fairness, privacy, contestability, and regulation concern the complete sociotechnical system, not only the model.

> **Key terms.** artificial intelligence; machine learning; supervised learning; unsupervised learning; neural network; language model; token; training; validation; test set; calibration; uncertainty; explainability; interpretability; human-in-the-loop; automation bias; fairness; distribution shift; model card; AI Act.

16.11 Exercises and worked solutions

Exercise. Why is a high test accuracy insufficient for deploying an AI appointment assistant?

Worked solution. The test distribution may not match deployment; errors may be unevenly distributed; the output may be poorly calibrated; the task may have asymmetric harms; staff may misuse the prediction; and citizens may lack contestability. System evaluation must include human and organisational outcomes.

Exercise. What is data leakage in model evaluation?

Worked solution. It occurs when information unavailable at prediction time, or information influenced by the target or test outcome, enters training or feature construction. It can produce unrealistically strong performance. The data-generation timeline must be examined.

Exercise. Give one appropriate human-control mechanism.

Worked solution. A staff member may see the model's recommendation, supporting evidence and uncertainty, choose an action, record a reason for overriding, and route contested cases to a human review process. Whether this is adequate depends on workload, authority, and error consequences.

16.12 Further reading

Human-centred AI connects interaction, classification, model development, evaluation, trust, fairness, explainability, and societal implications. For foundations, see **?**], **?**], and **?**]; for human–AI interaction, see **?**].

Chapter 17

Entrepreneurship and digital products

17.1 A motivating problem

A team builds a polished appointment app and nobody adopts it. The failure may be distribution, a weak value proposition, procurement constraints, trust, switching costs, or the fact that the original problem was not costly enough to justify change. Entrepreneurship is disciplined learning under uncertainty, not motivational language.

> **What you will learn.** You should be able to formulate value propositions; conduct problem and customer research; segment users and buyers; design minimum viable experiments; reason about metrics, pricing, costs, revenue, distribution, platform effects, accessibility, privacy, and ethical failure; and distinguish product-market evidence from enthusiasm.

17.2 The five-minute explanation

A product is an offer embedded in a context of users, buyers, alternatives, routines, regulation, and distribution. A value proposition states who has which problem, why existing alternatives are insufficient, and what change the product may create. It is a hypothesis until behaviour or credible evidence supports it.

For CivicQueue, the user may be a citizen, the buyer a municipality, the daily operator a service centre, and the data controller a public organisation. Their incentives and success criteria differ. A product decision must identify whose problem is being tested.

17.3 Problem discovery

Interview for recent behaviour and consequences, not only opinions about a future product. Observe current workarounds. Quantify frequency and cost cautiously. Separate a stated desire from willingness and ability to adopt. Map competitors and non-consumption: a paper form, telephone call, spreadsheet, or doing nothing may be the incumbent alternative.

17.4 Minimum viable experiments

An MVP is not necessarily a small version of the final software. It is the smallest experiment that tests a critical assumption. A concierge workflow, clickable prototype, landing page, manual service, or integration mock can be more informative than a production build.

Write each experiment with:

1. hypothesis;
2. target participant or buyer;
3. intervention;
4. observable behaviour;
5. threshold or decision rule;
6. duration and cost;
7. ethical and privacy safeguards.

Avoid vanity metrics such as page views when the risk concerns repeated use, payment, completion, or organisational adoption.

17.5 Business models and economics

A business model describes value creation, delivery, and capture. Record fixed costs, variable costs, acquisition costs, support, infrastructure, compliance, and opportunity cost. Pricing can be per user, per transaction, subscription, licence, usage, or public procurement. A price is not proof of value; willingness to pay depends on budget, authority, alternatives, and switching cost.

Platform products face network effects and governance problems. More participants may increase value, but also spam, concentration, lock-in, or unequal visibility. Accessibility and privacy can be competitive and ethical requirements rather than optional features.

17.6 Distribution and product-market fit

Distribution is how a product reaches a context of use. B2B and public-sector distribution may involve procurement frameworks, partnerships, pilots, trust, references, and long sales cycles. Product-market fit is not a single universally valid metric. Look for repeated use, retention, referrals, renewal, low-friction adoption, and evidence that the product solves a meaningful problem for a defined segment.

17.7 Ethics and failure

Growth incentives can reward addictive engagement, surveillance, exclusion, or dark patterns. A product experiment should specify who may be harmed and how participation can stop. A failed experiment is useful when it falsifies an assumption and prevents larger waste. A successful experiment may still be locally valid and not scalable.

17.8 Project application

Include a hypothesis table, interview or observation protocol, experiment log, value proposition, alternative analysis, basic cost model, distribution plan, and ethics section. When presenting the work, explain which assumption is riskiest, what result would cause a pivot, and why the chosen metric represents meaningful behaviour.

17.9 Summary and key terms

Entrepreneurship tests a product system under uncertainty. Discovery begins with problems and alternatives. MVPs test assumptions rather than merely reduce features. Business models include costs, buyers, distribution, regulation, and operations. Metrics should represent behaviour tied to a hypothesis. Ethical design includes what happens when the product succeeds or fails.

> **Key terms.** problem discovery; value proposition; customer; user; buyer; segment; MVP; hypothesis; experiment; metric; pricing; revenue; cost structure; distribution; product-market fit; platform; network effect; switching cost; ethical design.

17.10 Exercises and worked solutions

Exercise. Design a non-software MVP for CivicQueue.

Worked solution. A staff member could manually offer a rescheduling form and process requests using a controlled spreadsheet for one service and one week. Measure completion, time, conflicts, citizen understanding, and staff burden. This tests demand and workflow fit before building integration. Personal data would require appropriate safeguards or synthetic records.

Exercise. Why is a user not always the buyer?

Worked solution. A citizen may use a public-service system while a municipality procures it, managers approve it, staff operate it, and another organisation hosts it. Their incentives, risks, and decision rights differ. A product must satisfy the adoption system, not only the end user.

Exercise. Give one vanity metric and one stronger metric for a rescheduling product.

Worked solution. Number of page views is a weak vanity metric. The proportion of eligible users who complete a rescheduling request without staff re-entry, together with error and exclusion rates, is closer to the hypothesised value, though it still requires careful interpretation.

17.11 Further reading

Treat entrepreneurship as hypothesis testing and connect it to the research methods in Chapter 15. A promising experiment is evidence about an assumption, not proof of a business.

Chapter 18

The master's thesis: a defensible contribution

18.1 A motivating problem

A thesis proposal says, "We will build an innovative AI platform that transforms public services". It contains ambition but no feasible boundary, research question, method, contribution, or evaluation. A master's thesis earns its strength through delimitation: a claim small enough to investigate and important enough to matter.

> **What you will learn.** You should be able to select and delimit a feasible thesis problem; build a conceptual framing and literature review; choose a research or design method; plan an artefact and evaluation; distinguish contribution types; report results and limitations; manage scholarly collaboration; and present a defensible argument.

18.2 The five-minute explanation

A thesis is a sustained argument. It tells the reader what problem was investigated, why it matters, what is already known, what was done, what was found, what can be concluded, and what remains uncertain. A software artefact may be central, but the thesis must explain what knowledge the work contributes and how evidence supports that claim.

18.3 From topic to research question

Start with a domain problem and an accessible source of evidence. Delimit by population, context, task, system boundary, time, data, and contribution. A feasible question often has the form:

> How does [intervention or design] affect or support [construct or activity] for [population] in [context], and under which conditions?

The wording should match the design. If there is no comparison, do not imply a causal effect. If the study concerns one organisation, delimit transfer.

18.4 Literature and conceptual framing

The literature review should justify definitions, mechanisms, methods, and gap. A gap may be empirical, theoretical, methodological, design-oriented, or contextual. It need not mean that nobody has ever studied the topic. The contribution can be a careful replication, synthesis, boundary condition, or evaluated artefact.

Build a matrix with source, question, theory, method, data, finding, limitation, and relevance. Cite primary authors for concepts and original empirical studies where possible.

18.5 Method and artefact

Possible thesis designs include:

- an empirical study of use, adoption, or organisational change;
- a design-science project that builds and evaluates an artefact;
- a theoretical investigation of an interactive-system problem;
- a mixed-methods study integrating qualitative context and quantitative outcomes;
- a systematic or scoping review with a transparent protocol.

An artefact can be a software system, interaction design, model, method, dataset, or architecture. Specify the build process, requirements, design rationale, implementation limits, evaluation, and reproducibility.

18.6 Contribution types

Table 18.1: Distinguishing thesis contribution types.

Contribution	What it adds	Evidence required
Empirical	A finding about a defined context or population	Data, design, analysis, uncertainty, limitations
Theoretical	A concept, relationship, mechanism, or boundary condition	Conceptual argument and engagement with literature
Methodological	A procedure or measurement approach	Rationale, implementation, comparison or evaluation
Artefact/design	A system, prototype, architecture, or design knowledge	Requirements, construction, rationale, evaluation
Replication	A reasoned reproduction or extension of prior work	Faithful protocol, comparison, deviations
Synthesis	An integrated account of prior evidence	Search, selection, appraisal, and synthesis method

A thesis can contain several contribution types, but name which is central. Novelty is not a substitute for validity.

18.7 Results, discussion, and limitations

Results report what the analysis produced. Discussion interprets the result in relation to the question and literature. Limitations explain how the design constrains inference. A limitation should not be a ritual list; it should say which claim is weakened and in what direction.

A defensible conclusion uses calibrated verbs: *describes*, *is consistent with*, *suggests*, *supports under these conditions*, or *does not establish*. Avoid *proves* unless the domain and proof standard genuinely warrant it.

18.8 Planning and supervision

Create a reverse plan from the final delivery and presentation. Reserve time for ethics, access, recruitment, data cleaning, implementation debt, analysis, writing, references, visual QA, and contingency. Supervision is a scholarly collaboration, not an outsourced project-management function. Bring concrete questions, evidence of progress, and a record of decisions.

18.9 Writing and presenting the thesis

A clear chapter structure is an argument structure: introduction, related work, method, design or system, results, discussion, conclusion, references, appendices. Figures should be introduced, captioned, interpreted, and readable in print. Code should be versioned and runnable or its irreproducibility explained.

When presenting a thesis, practise answers to:

1. What is the exact research question?
2. What is the central contribution?
3. Why is the method appropriate?
4. What alternative explanation is most serious?
5. Which design decision would you change with more time?
6. What does the thesis not establish?

18.10 Project application

The thesis should connect the full progression of this book: problem, model, requirements, design, implementation, data, evaluation, organisation, and claim. Its scope, contribution, method, and evidence should be clear enough that a critical reader can reconstruct the path from question to conclusion.

18.11 Summary and key terms

A master's thesis is a bounded argument supported by theory, method, artefact or data, analysis, and reflection. Delimitation is a strength. Contribution types differ in evidence requirements. Results, interpretation, and limitation should be separate. A strong presentation demonstrates understanding and judgment, not only a polished final product.

> **Key terms.** thesis; delimitation; research question; literature review; conceptual framework; design science; artefact; empirical contribution; theoretical contribution; methodological contribution; replication; synthesis; result; discussion; limitation; reproducibility; presentation.

18.12 Exercises and worked solutions

Exercise. Why is "build an innovative AI platform" not yet a thesis question?

Worked solution. It names an artefact and an evaluative adjective but not a phenomenon, population, context, comparison, method, or contribution. A bounded version might ask how a specified AI interaction affects a defined task for a defined group under specified conditions, with an explicit evaluation design.

Exercise. What is the difference between a limitation and a failure?

Worked solution. A limitation is a condition that restricts what can be inferred, even when the study is competently executed. A failure is a departure from the planned or required process that may invalidate part of the evidence. A limitation can be responsibly designed; a failure must be analysed and disclosed.

Exercise. What should a supervisor meeting produce?

Worked solution. It should produce clarified questions, decisions, risks, next actions, and a record of unresolved issues. The student remains responsible for the argument and should not treat disagreement as a substitute for methodological reasoning.

18.13 Further reading

Use the research-methods chapter as the methodological foundation. For case-study reporting, see **?**]; for experimentation, see **?**].

Part V

Integration projects

Chapter 19

Complete worked case: CivicQueue

19.1 Why a complete case matters

Most tutorials show a function, a diagram, or a database query in isolation. Real projects fail at the boundaries between them. This case follows one bounded intervention through problem discovery, requirements, user research, domain modelling, interface design, architecture, implementation, persistence, testing, deployment, evaluation, and research reflection.

CivicQueue is fictional. It is not a prescription for public-sector systems and contains no real citizen data.

19.2 Problem discovery

The starting observation is that people who need to change an appointment often call the centre, wait, and sometimes fail to reach the right person. Staff report repeated manual entry and late changes. These observations are hypotheses about a process, not proof that a web application is the best intervention.

The team conducts:

- short observations of the current booking and rescheduling process;
- interviews with citizens, front-desk staff, and a service manager;
- document analysis of appointment rules and privacy procedures;
- a review of existing calendar and case-system interfaces.

The team records contradictions instead of averaging them away. Citizens value flexibility; staff value control over scarce slots; managers value predictable utilisation; data-protection officers value minimisation and auditability.

19.3 Stakeholder analysis and delimitation

The first release is deliberately bounded:

- one service centre;
- one service type;

- authenticated citizens who already have an appointment;
- rescheduling to an available future slot;
- staff visibility of changes;
- a human and telephone fallback.

Out of scope are automated eligibility decisions, cross-municipality identity, emergency appointments, and production payment or notification integration. Delimitation is an engineering and research decision.

19.4 Requirements and traceability

The project writes requirements in plain language and adds acceptance criteria.

Table 19.1: CivicQueue requirements and evidence.

ID	Requirement	Acceptance criterion	Evidence
R1	A citizen can view their appointment	Only the authenticated owner sees the appointment details and status	request test
R2	A citizen can request a future available slot	The server rejects unavailable or past slots without changing the current appointment	domain and integration tests
R3	A slot cannot be double-booked	Concurrent or repeated attempts leave at most one active reservation	database constraint and transaction test
R4	A user receives clear feedback	Success, validation failure, conflict, and service failure are distinguishable and accessible	usability and accessibility review
R5	Staff can review changes	A staff view shows the old and new status with an audit event	query and workflow test
R6	A non-digital fallback exists	The interface gives a telephone or staff route without punishing the user	user research and process review

19.5 Domain model

The key distinction is between an available slot and an appointment. A slot has service, location, start time, end time, and capacity. An appointment links a citizen to a slot and has a lifecycle. A rescheduling action either creates a new confirmed appointment and releases the old one, or leaves the old state unchanged.

The invariant is:

$$\forall s \in \text{Slots}, \quad \#\{\text{active appointments for } s\} \leq 1.$$

The quantifier $\forall$ means “for every”; the expression says that no slot has more than one active appointment. The database constraint and application transaction must work together to enforce it.

19.6 Interaction design

The primary task flow is:

1. view current appointment;
2. select “change time”;
3. inspect available slots with date, time zone, service, and location;
4. choose a slot;
5. review consequences;
6. confirm;
7. receive success or a conflict with a recovery route.

The interface does not silently cancel the old appointment when the new reservation fails. The success message identifies the new time. The conflict message explains that the slot was taken and returns the user to available options.

The team tests a low-fidelity prototype before implementing the visual design. It checks language, grouping, back navigation, error recovery, and whether users understand that a change is not complete until confirmation.

19.7 Architecture

The system uses a modular monolith:

- presentation: HTML templates and a small JSON API;
- application service: use-case orchestration and authorisation checks;
- domain: appointment state transitions and value validation;
- persistence: parameterised SQL and transactions;
- external boundary: an adapter for future identity and notification providers.

This architecture is adequate for the bounded case and avoids introducing distributed-system costs before the domain boundary is known.

19.8 Implementation

The repository contains a runnable Flask application. Run its tests before changing it. The application uses a local SQLite database for teaching. Production identity, TLS, secret management, backups, concurrency configuration, and deployment hardening are deliberately outside the demo’s safety claims.

A request to reschedule passes through:

$$\text{form} \rightarrow \text{route} \rightarrow \text{service} \rightarrow \text{transaction} \rightarrow \text{response} \rightarrow \text{feedback}.$$

The route does not decide whether a cancelled appointment can be completed. The domain service does. The database does not know the user’s intention; it enforces structural constraints.

19.9 Database and transaction

The seed data contains citizens, services, slots, and appointments. A reservation transaction:

1. begins a transaction;
2. checks that the caller owns the current appointment;
3. checks that the target slot is future and available;
4. inserts the new appointment;
5. changes the old appointment to cancelled or rescheduled;
6. writes an audit event;
7. commits or rolls back as one unit.

A unique constraint protects the slot even if two requests pass the application-level availability check. The service maps a uniqueness conflict to an explicit user-facing response.

19.10 Testing

The test plan follows the risk:

- unit tests for legal and illegal appointment transitions;
- repository tests for constraints and parameterised queries;
- request tests for ownership and HTTP status mapping;
- integration tests for successful and conflicting rescheduling;
- keyboard and screen-reader checks for the primary task;
- exploratory tests for refresh, back navigation, stale slots, and timeouts.

A green test run supports the tested claims. It does not establish deployment readiness or population-level service benefit.

19.11 Deployment and operation

For a real deployment, document the runtime, environment variables, secrets, database migration, TLS, backups, health check, logs, alert owner, and rollback. The demonstration app can run locally; that is evidence of local reproducibility, not evidence of availability or security in a public service.

19.12 Evaluation

The team separates three questions:

1. Can intended users complete the rescheduling task?
2. Does the integrated system preserve correctness and privacy?

3. Does using the service improve the organisation's outcomes under a specified implementation?

A usability study may answer the first. Tests and security review address the second. The third requires organisational data and a design capable of addressing alternative explanations.

A bounded formative evaluation recruits participants who represent relevant tasks and access needs. It measures task completion, error recovery, perceived clarity, and staff burden. It reports who participated, what was simulated, and what was not tested.

19.13 Reflection and research contribution

The artefact contribution is a tested design and implementation for a bounded rescheduling workflow. A possible empirical contribution would require a separate study. A process contribution might document how domain modelling and interaction evaluation changed the implementation. The team should not call a prototype a validated public-service intervention without evidence of use and outcomes.

19.14 Written account and presentation checklist

1. State the problem and why it is worth addressing.
2. Explain the boundary and out-of-scope decisions.
3. Link requirements to models, code, and tests.
4. Show the request path and transaction invariant.
5. Demonstrate success and failure paths.
6. Explain accessibility, privacy, security, and organisational consequences.
7. State what the evaluation can and cannot establish.

19.15 Summary

CivicQueue works as a case because every technical decision is connected to a problem, user, organisation, and claim. The system is not finished when the route returns 200. It is finished for a stated scope when its behaviour, assumptions, risks, evidence, and limitations are understandable.

> **Key terms.** vertical slice; delimitation; traceability; domain invariant; transaction; active appointment; modular monolith; formative evaluation; fallback; artefact contribution; organisational outcome.

19.16 Exercises and worked solutions

Exercise. Which requirement is most directly protected by a database uniqueness constraint?

Worked solution. R3, the requirement that a slot cannot be double-booked. The constraint provides protection at the persistence boundary, while the application transaction and user-facing conflict response make the invariant meaningful in the complete system.

Exercise. Why is the human fallback part of the system rather than a separate service issue?

Worked solution. It affects accessibility, adoption, error recovery, workload, and the consequences of digital failure. Excluding it from the boundary would hide a condition of service use.

Exercise. What additional evidence would be needed before claiming that CivicQueue reduces missed appointments?

Worked solution. A defined outcome and comparison, data over an appropriate period, an implementation with known adoption, and a design addressing seasonality, staffing, selection, and other changes. A formative usability test alone is insufficient.

Chapter 20

Integration projects and worked solutions

20.1 How to use these projects

The following projects are opportunities to bring the book's ideas together. Each requires a problem formulation, model, implementation or design, evidence, reflection, and a clear presentation. Choose one that matches your interests, but retain the full chain from problem to claim.

20.2 Project A: an agent-based public-service model

Problem. Under what conditions can a small change in appointment flexibility affect queue pressure in a simplified service-centre model?

Deliverables. Define agents, state variables, transition rules, scheduling assumptions, outcome measures, random seeds, sensitivity analysis, code tests, and a limitation section. Do not claim to predict a real municipality from the model.

Worked solution outline. Represent citizens as agents with arrival, service, and rescheduling states. Represent staff as capacity-constrained servers. Compare a baseline rule with a flexibility rule under the same seeds and demand scenarios. Verify state transitions and conservation of appointments. Validate only the computational implementation and argue cautiously about the model's relevance.

20.3 Project B: a complete vertical slice

Problem. Build one accessible workflow for a community organisation to request and confirm appointments.

Deliverables. Stakeholder analysis, user story and acceptance criteria, domain model, interface prototype, HTML/CSS/JavaScript or server-rendered client, API, database schema, tests, threat model, deployment instructions, and a recorded decision log.

Worked solution outline. Start with one task. Implement a thin route, a domain service, and a parameterised repository. Add the success path and one conflict path before expanding features. Demonstrate how the same domain invariant is protected in tests and in the

database. Evaluate task completion and error recovery with a stated protocol.

20.4 Project C: comparative interface evaluation

Problem. Does a revised appointment form reduce interpretation errors for a defined group and task?

Deliverables. Construct definition, hypotheses, prototype versions, sampling plan, task script, measures, consent, analysis plan, results, validity discussion, and accessible materials.

Worked solution outline. Define an error before recruitment. Randomise order if a within-participant comparison is used, or justify a between-participant design. Report denominators and uncertainty. Interpret a difference only within the task and sample. Include qualitative observations that explain how errors occurred, while keeping exploratory findings separate from pre-specified outcomes.

20.5 Project D: human-centred AI assistant

Problem. Design and evaluate an assistant that helps staff find relevant policy information without making final eligibility decisions.

Deliverables. Human task analysis, data and privacy map, baseline search workflow, model or retrieval design, evaluation split, calibration or uncertainty discussion, interface explanation, correction and escalation path, subgroup risk analysis, model documentation, and system evaluation.

Worked solution outline. Compare the assistant to keyword search or a curated index. Measure retrieval quality, time, error types, and staff verification. Do not count fluent text as accuracy. Require citation or source display, visible uncertainty, and a route for reporting errors. Evaluate the combined human–AI workflow and document when the simpler baseline is preferable.

20.6 Project E: digital product experiment

Problem. Test whether a defined group would adopt a service that reduces a costly coordination task.

Deliverables. Problem interviews, segment, value proposition, alternative map, experiment, threshold, cost model, distribution route, privacy and accessibility plan, result, pivot decision, and reflection.

Worked solution outline. Use a manual or low-fidelity experiment to test the riskiest assumption. Observe behaviour rather than asking only whether the idea sounds good. Track the denominator and recruitment channel. Treat a negative result as information about the segment, problem, intervention, or distribution, not as a verdict on the team's worth.

20.7 Cross-project review rubric

Table 20.1: A compact rubric for integration work.

Dimension	Evidence of strong work
Problem and delimitation	The problem is evidenced, bounded, and not silently replaced by a preferred technology.
Conceptual reasoning	Terms, models, assumptions, and mechanisms are defined and used consistently.
Technical quality	The implementation or prototype is runnable, tested, secure within scope, and explained.
Human and organisational fit	Users, accessibility, privacy, work routines, and power are considered with evidence.
Evaluation	The method fits the claim; measures, uncertainty, validity, and limitations are explicit.
Reflection	Decisions, changes, group process, failures, and trade-offs are analysed rather than narrated.
Communication	The written account, figures, demonstration, and presentation form one coherent argument.

20.8 Final synthesis

The book's topics are not separate islands. Computational thinking supplies representations and transitions. Programming makes procedures executable. Algorithms organise time and space. Software engineering coordinates change. Modelling makes boundaries and responsibilities discussable. Databases preserve state. Web development connects people and services. HCI studies action and interpretation. Information-systems analysis follows technology into organisations. Research disciplines claims. Human-centred AI extends the same questions to probabilistic systems.

A graduate who can build but not evaluate is incomplete. A graduate who can critique but not implement is also incomplete. The professional standard is the ability to move between abstraction and practice while keeping assumptions visible.

20.9 Questions for a critical presentation

1. What problem did you choose, and what evidence shows that it is real?
2. Which abstraction or model made the problem tractable?
3. Which invariant or requirement does the system protect?
4. Which design alternative did you reject and why?
5. What happens under failure, misuse, exclusion, or organisational resistance?
6. What does your evaluation support?
7. What does it not support?
8. What would you do differently with more time or evidence?

20.10 Final checklist

Before sharing or submitting the work, run the code, rebuild the database from an empty state, execute the tests, inspect every figure, check links and citations, remove secrets and personal data, verify accessibility of the interface and PDF, and ask another person to reproduce the setup. Then read the conclusion as a critical reader: is every strong sentence supported by the method and evidence?

Appendix A

Mathematics and logic bridge

A.1 Propositions and logical connectives

A proposition is a statement that is either true or false in a given interpretation. If P and Q are propositions, then $P \wedge Q$ means both are true, $P \vee Q$ means at least one is true, and $\neg P$ means not P. The implication $P \Rightarrow Q$ is false only when P is true and Q is false. It does not mean that P causes Q.

For example, let P be “the slot is available” and Q be “the reservation may proceed”. A rule may state $P \wedge A \Rightarrow Q$, where A represents authorisation. The system must not silently drop either condition.

A.2 Predicates and quantifiers

A predicate is a statement whose truth depends on variables. $\forall x \in X\, P(x)$ means that P holds for every element of X. $\exists x \in X\, P(x)$ means that at least one element satisfies P. The appointment invariant in Chapter 19 uses a universal quantifier because it must hold for every slot.

A.3 Sets, functions, and relations

Set operations include union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, and Cartesian product $A \times B$. A relation is a subset of a Cartesian product. A function is a relation with exactly one output per input. In data modelling, ask whether a relationship is one-to-one, one-to-many, or many-to-many.

A.4 Sequences and sums

A sequence is an ordered list $(a_1, a_2, \ldots, a_n)$. The notation

$$\sum_{i=1}^{n} a_i$$

means add the values a_i for indices from 1 through n. The arithmetic mean is

$$\bar{a} = \frac{1}{n} \sum_{i=1}^{n} a_i.$$

The mean is sensitive to extreme values and is meaningful only for a defined measurement scale and population of observations.

A.5 Proof ideas

A direct proof starts with assumptions and derives the conclusion. A proof by contradiction assumes the conclusion is false and derives an impossibility. Induction uses a base case and a step from n to $n + 1$. In software, a loop invariant plays a similar role: it links local execution to a global correctness claim.

A.6 Probability

A probability model assigns values from 0 to 1 to events. If events A and B are mutually exclusive, $P(A \cup B) = P(A) + P(B)$. Conditional probability is

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

when $P(B) > 0$. The notation means the probability of A given that B occurred. Confusing $P(A \mid B)$ with $P(B \mid A)$ is a common error.

Expected value for a discrete variable is

$$E[X] = \sum_{x} xP(X = x).$$

It is a long-run average under the specified distribution, not necessarily an outcome that can occur in one run.

A.7 Vectors and matrices

A vector is an ordered tuple of numbers. A matrix is a rectangular array. In machine learning, x may be a feature vector and $w^\top x$ a weighted sum. Matrix dimensions must agree: if w and x each have d components, their dot product is $\sum_{j=1}^{d} w_j x_j$.

A.8 Units and dimensional reasoning

Keep units visible. A rate of 2 appointments per hour multiplied by 3 hours gives 6 appointments. Adding 2 hours to 3 appointments is meaningless. Dimensional checks catch some implementation and model errors before execution.

Appendix B

Python, Git, and the command line

B.1 Create a reproducible Python environment

From the project root:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r examples/webapp/requirements.txt
python -m unittest discover -s examples -p 'test_*.py' -v
```

On Windows PowerShell, activation uses `.venv\Scripts\Activate.ps1`. The environment isolates project dependencies; it does not make untrusted code safe.

B.2 Run and inspect code

Use `python path/to/file.py` to run a script. Use `python -m compileall` to check syntax without executing application behaviour. Prefer modules with functions that can be imported and tested rather than scripts whose only interface is printing.

B.3 Git essentials

```
git init
git status
git add path/to/file
git commit -m "Describe the change"
git log --oneline --decorate
git diff
git switch -c feature/rescheduling
git switch main
git merge feature/rescheduling
```

A commit is a snapshot and message, not a guarantee of quality. Do not commit passwords, API keys, private data, generated virtual environments, or local databases. Use a reviewed ignore file.

B.4 Debugging loop

Reproduce the problem with the smallest input. Read the complete error message. Identify the first violated assumption rather than patching the last symptom. Add a regression test. Change one thing. Run the relevant test and the complete suite. Record the explanation in the issue or decision log.

B.5 Code review questions

1. Does the change satisfy a stated requirement?
2. Are inputs validated at the boundary?
3. Are permissions checked on the server?
4. What happens on duplicate, stale, missing, or malicious input?
5. Are tests meaningful and independent?
6. Does the change affect accessibility, privacy, performance, or operations?
7. Is the documentation sufficient for another student to run it?

B.6 Command-line habits

Use a clear working directory, quote paths with spaces, inspect before deleting, and keep generated output separate from source. A shell command can have side effects; read it before running it. The build instructions in the repository are the authoritative sequence for this book.

Appendix C

Reference tables

C.1 Common complexity classes

Class	Typical example	Interpretation
$O(1)$	hash lookup, under assumptions	bounded by input size in the model
$O(\log n)$	binary search on sorted indexed data	repeated reduction of search space
$O(n)$	linear scan	work proportional to input size
$O(n \log n)$	comparison sorting families	near-linear times logarithmic growth
$O(n^2)$	nested pair comparison	all or many pairs

These are growth descriptions, not benchmark results.

C.2 Selected HTTP status meanings

200	successful response
201	resource created
204	successful response with no content
400	malformed or invalid request
401	authentication required or failed
403	authenticated but not authorised
404	resource not found
409	conflict with current state
422	semantically invalid content, where used by the API
429	rate limit exceeded
500	unexpected server failure
503	service temporarily unavailable

The exact API contract should define which status is used for which condition.

C.3 Evaluation-method selection

Question	Useful method
Can experts identify interaction violations?	heuristic evaluation
Can a new user discover the task?	cognitive walkthrough or user study
What do people do in context?	observation or contextual inquiry
How do people describe trust or confusion?	interview or think-aloud
Does one version differ under a task?	comparative study with pre-defined outcomes
Does an organisational outcome change?	longitudinal or quasi-experimental design with process evidence

C.4 Security review prompts

Identify assets, actors, trust boundaries, input validation, authorisation, secrets, logging, dependencies, rate limits, recovery, privacy, accessibility, and misuse. A checklist is a starting point; the threat model determines relevance.

Appendix D

Glossary

Abstraction A selective representation that omits detail for a purpose.

Acceptance criterion An observable condition used to decide whether a requirement is satisfied.

Accessibility The extent to which people with diverse abilities can perceive, operate, understand, and use a system.

ACID Atomicity, consistency, isolation, and durability properties of transactions.

Actor A role external to a system boundary that participates in an interaction.

Agile A family of principles and practices for iterative learning and delivery under change.

Algorithm A specified procedure for transforming inputs into outputs.

API An interface through which software components communicate.

Authorisation The decision about what an authenticated actor may do.

Authentication Establishing or asserting the identity of an actor.

Bias A systematic distortion or unequal pattern introduced by data, measurement, design, or use.

Class A definition from which objects with related structure or behaviour may be created.

Client A component that requests a service from another component.

Complexity A description of resources such as time or space as input size changes.

Constraint A condition that limits or validates possible states or actions.

Construct An abstract concept represented through measurement or design.

Coupling The degree to which components depend on one another.

Data Recorded distinctions or representations interpreted in a context.

Database A managed system for storing, querying, and updating structured information.

Digitalisation Organisational or social change enabled by digital technologies.

Encapsulation Protecting representation behind a controlled interface.

Evaluation A systematic judgement of an artefact, design, process, or claim against criteria.

Exception A mechanism for transferring control when normal execution cannot continue.

Function A mapping from inputs to outputs; in programming it may also have effects.

Graph A structure of vertices and edges.

HCI Human–computer interaction: study and design of interactive systems in use.

HTTP A protocol for communication between clients and servers on the web.

Information system A sociotechnical arrangement that produces and uses information.

Invariant A proposition that remains true across specified operations or states.

Iteration Repetition that may incorporate learning or changed decisions.

Key An attribute or set of attributes used to identify a tuple.

Machine learning Methods that estimate patterns or functions from data.

Model A selective representation of a phenomenon, system, or design.

Normalisation Relational decomposition guided by dependencies and anomaly reduction.

Object An entity with identity, state, and behaviour in a program.

Polymorphism The ability to use a common interface with varying implementations.

Prototype An artefact made to explore or communicate a design.

Requirement A needed capability, condition, or constraint in context.

Research contribution A defensible addition to knowledge, method, theory, design, or evidence.

REST An architectural style for networked systems based on constraints including representations and stateless interactions.

Schema A formal description of data structure and constraints.

Security Protection of assets against threats under specified assumptions.

Stakeholder A person, group, or institution affected by or able to affect a system.

State The information needed to describe a system at a time for a purpose.

Technical debt A present design choice that creates future cost or reduced options.

Transaction A coordinated unit of database work with defined consistency behaviour.

Usability Effectiveness, efficiency, and satisfaction in a specified context of use.

Validation Judging whether a model or artefact is adequate for a purpose.

Verification Checking whether an implementation conforms to a specification.

Bibliography

Name index

A searchable PDF index can be generated from the source with the standard LaTeX indexing tools. The principal authors and concepts are also listed in the glossary and bibliography.

Subject index

The table of contents, key terms, and cross-references provide the subject index for this compact edition.